SVKM's NMIMS
Mukesh Patel School of Technology Management & Engineering (MPSTME)
Shirpur Campus

B.Tech. Information Technology
Course: Design Experience
Academic Year 2026–2027

DESIGN EXPERIENCE PROJECT REPORT

DEEP LEARNING-BASED CYBERSECURITY THREAT DETECTION USING NETWORK TRAFFIC

Submitted by

Sr. No.	Name of Student	Roll No.
1	Adivya Kumar Singh	A237
2	Sarthak Salot	A228
3	Dhruvit Patel	A212
4	Vaibhav Raghuvanshi	A218
5	Aryan Raghuvanshi	A214
6	Vedansh Kanoongo	A253
7	Yuvraj Kale	A210

Under the guidance of
Faculty Guide: Vinod Waikar

PROJECT INFORMATION

Project title	Deep Learning-Based Cybersecurity Threat Detection Using Network Traffic
Course	Design Experience
Faculty guide	Vinod Waikar
Institute	SVKM's NMIMS, Mukesh Patel School of Technology Management & Engineering (MPSTME), Shirpur Campus
Programme	B.Tech. Information Technology
Academic year	2026–2027
Team size	7 members
Dataset	CIC-YNU-IoTMal Dataset 2026 (x86 PCAP network-traffic data)
Records used	455,641 raw records, 42 columns; 424,110 records after removing Unknown labels; 39 input features
Experiment 1	Five-class malware-family classification: test accuracy 32.72%
Experiment 2	Binary Benign-versus-Malicious classification: test accuracy 98.02%, ROC-AUC 0.9980
Tools	Python, Google Colab, Pandas, NumPy, Scikit-learn, TensorFlow, Keras, Matplotlib, Seaborn, ipywidgets
Saved model	cyber_threat_detector.keras

TABLE OF CONTENTS

1. INTRODUCTION....1
1.1 Background....1
1.2 Motivation....1
1.3 Problem Statement....1
1.4 Aim....1
1.5 Objectives....1
1.6 Scope of the Project....2
1.7 Project Workflow....2
1.8 Organization of the Report....3
2. SYSTEM ANALYSIS AND PROJECT REQUIREMENTS....4
2.1 Existing Problem....4
2.2 Proposed System....4
2.3 System Objectives....4
2.4 Functional Requirements....4
2.5 Non-Functional Requirements....5
2.6 Hardware Requirements....5
2.7 Software Requirements....5
2.8 Technology Stack....6
2.9 System Inputs....6
2.10 System Outputs....6
2.11 Development Environment....6
3. DATASET DESCRIPTION....7
3.1 Dataset Overview....7
3.2 CIC-YNU-IoTMal Dataset....7
3.3 Why This Dataset Was Selected....7
3.4 X86 PCAP Dataset....8
3.5 Dataset Dimensions....8
3.6 Dataset Attributes....8
3.7 Target Variable....9
3.8 Malware/Traffic Categories....9
3.9 Class Distribution....10
3.10 Dataset Characteristics....10
4. DATA PREPROCESSING....11
4.1 Data Loading....11
4.2 Initial Dataset Inspection....11
4.3 Data Cleaning....11
4.4 Handling Unknown Labels....11
4.5 Removal of Unnecessary Information....11
4.6 Missing Value Analysis....12
4.7 Infinite Value Analysis....12
4.8 Feature Preparation....13
4.9 Target Label Preparation....13
4.10 Train-Test Split....13
4.11 Feature Scaling....14
4.12 Final Dataset Dimensions....14
4.13 Preprocessing Summary....14
5. EXPLORATORY DATA ANALYSIS....16
5.1 Purpose of EDA....16
5.2 Class Distribution....16
5.3 Feature Analysis....16
5.4 Observations....16
5.5 Impact of Class Imbalance....17
6. PROPOSED METHODOLOGY....18
6.1 Overall Methodology....18
6.2 System Workflow....18
6.3 Data Processing Pipeline....18
6.4 Multiclass Classification Approach....18
6.5 Binary Classification Approach....19

6.6 Model Training..... 19
6.7 Model Evaluation..... 19
6.8 Interactive Detection..... 19
7. DEEP LEARNING MODEL DEVELOPMENT..... 20
7.1 Neural Network Architecture..... 20
7.2 Input Layer..... 20
7.3 Hidden Layers..... 20
7.4 Dropout Layers..... 20
7.5 Output Layer..... 21
7.6 Activation Functions..... 21
7.7 Loss Functions..... 21
7.8 Optimizer..... 21
7.9 Training Parameters..... 21
7.10 Model Summary..... 21
7.11 Training Performance..... 22
8. EXPERIMENT 1 — MALWARE FAMILY CLASSIFICATION..... 23
8.1 Experiment Objective..... 23
8.2 Dataset Preparation..... 23
8.3 Model Configuration..... 23
8.4 Training Process..... 23
8.5 Training and Validation Results..... 23
8.6 Classification Report..... 25
8.7 Confusion Matrix..... 26
8.8 Performance Analysis..... 26
8.9 Challenges Observed..... 27
8.10 Decision to Reformulate the Problem..... 27
9. EXPERIMENT 2 — BINARY CYBER THREAT DETECTION..... 28
9.1 Experiment Objective..... 28
9.2 Binary Label Conversion..... 28
9.3 Class Distribution..... 28
9.4 Class Weighting..... 29
9.5 Model Architecture..... 29
9.6 Training Configuration..... 29
9.7 Training Performance..... 29
9.8 Classification Report..... 31
9.9 Confusion Matrix..... 32
9.10 ROC Curve..... 33
9.11 ROC-AUC Analysis..... 33
9.12 Overall Performance..... 33
10. INTERACTIVE CYBER THREAT DETECTOR..... 34
10.1 Purpose..... 34
10.2 Interface Design..... 34
10.3 Prediction Workflow..... 35
10.4 Sample Selection..... 35
10.5 Probability Calculation..... 35
10.6 Threshold Decision..... 35
10.7 Confidence Score..... 35
10.8 Benign Prediction..... 35
10.9 Malicious Prediction..... 36
10.10 Model Saving..... 36
11. RESULTS AND PERFORMANCE ANALYSIS..... 38
11.1 Summary of Results..... 38
11.2 Multiclass Results..... 38
11.3 Binary Results..... 38
11.4 Comparison of Experiments..... 38
11.5 Confusion Matrix Analysis..... 39
11.6 ROC-AUC Analysis..... 39
11.7 Key Findings..... 40
12. DISCUSSION..... 41
12.1 Interpretation of Results..... 41

12.2 Major Findings..... 41
12.3 Strengths..... 41
12.4 Practical Relevance..... 41
12.5 Lessons Learned During Development..... 41
13. CONCLUSION AND FUTURE SCOPE..... 43
13.1 Conclusion..... 43
13.2 Future Scope..... 43
14. REFERENCES..... 44

LIST OF FIGURES

Figure 1.1: Overall Project Workflow..... 3
Figure 3.1: Dataset Loading and Dataset Shape in Google Colab..... 8
Figure 3.2: Column Names of the Dataset..... 8
Figure 3.3: MalwareFamily Counts in the Raw Dataset..... 9
Figure 4.1: Removal of Unknown Labels, Dropping Hash and Arch, and Median Filling..... 12
Figure 4.2: Missing Values per Column Before Cleaning..... 12
Figure 4.3: Detection of Infinite Values in the Training and Testing Data..... 13
Figure 4.4: Verification of Missing and Infinite Values After Data Preprocessing..... 13
Figure 4.5: Train-Test Split Shapes and the Error Raised by the First Scaling Attempt..... 14
Figure 4.6: Feature Scaling and Final Training and Testing Shapes..... 14
Figure 5.1: Distribution of Malware Families After Cleaning..... 16
Figure 6.1: Proposed System Architecture..... 18
Figure 7.1: Deep Learning Model Architecture..... 20
Figure 7.2: Model Summary of the Five-Class Model (Experiment 1)..... 22
Figure 7.3: Model Summary of the Binary Model (Experiment 2)..... 22
Figure 8.1: Training and Validation Accuracy of the Five-Class Model..... 24
Figure 8.2: Training and Validation Loss of the Five-Class Model..... 24
Figure 8.3: Test Evaluation and Prediction Generation for the Five-Class Model..... 25
Figure 8.4: Classification Report of the Five-Class Model..... 25
Figure 8.5: Confusion Matrix of the Five-Class Model..... 26
Figure 9.1: Binary Classification Workflow..... 28
Figure 9.2: Binary Label Conversion and Class Counts..... 29
Figure 9.3: Training Log of the Binary Model..... 30
Figure 9.4: Training and Validation Accuracy and Loss of the Binary Model..... 30
Figure 9.5: Test Evaluation of the Binary Model..... 31
Figure 9.6: Classification Report of the Binary Model..... 31
Figure 9.7: Confusion Matrix of the Binary Model..... 32
Figure 9.8: ROC Curve of the Binary Model..... 33
Figure 10.1: Interactive Cyber Threat Detector Workflow..... 34
Figure 10.2: Interactive Cyber Threat Detector with a Benign Prediction..... 34
Figure 10.3: Text-Menu Version of the Detector (Earlier Version)..... 35
Figure 10.4: Benign Sample Tested with the detect_traffic Function..... 36
Figure 10.5: Malicious Prediction Shown in the Detector Output..... 36
Figure 10.6: Final Results Summary and Model Saving..... 37
Figure 11.1: Experiment Comparison..... 38

LIST OF TABLES

Table 2.1: Functional Requirements....4
Table 2.2: Non-Functional Requirements....5
Table 2.3: Hardware Requirements....5
Table 2.4: Software and Technology Stack....6
Table 3.1: Dataset Summary....7
Table 3.2: Important Dataset Attributes....8
Table 3.3: First Five Records of the Dataset (Selected Columns)....9
Table 3.4: Class Distribution of the Dataset....10
Table 4.1: Preprocessing Summary....14
Table 7.1: Training Parameters....21
Table 7.2: Model Architecture....21
Table 8.1: Class Distribution and Class Weights in Experiment 1....23
Table 8.2: Epoch-wise Training Results of Experiment 1....24
Table 8.3: Multiclass Classification Results (Test Set)....25
Table 8.4: Confusion Matrix Values of the Five-Class Model....26
Table 9.1: Class Distribution in the Binary Experiment....28
Table 9.2: Epoch-wise Training Results of Experiment 2....30
Table 9.3: Performance of the Binary Classification Model (Test Set)....31
Table 9.4: Binary Confusion Matrix....32
Table 11.1: Comparison of Experiments....38
Table 11.2: Error Analysis of the Binary Model (Calculated from the Confusion Matrix)....39

1. INTRODUCTION

1.1 Background

Computer networks now carry traffic from a very large number of small, low-cost devices such as routers, cameras and other Internet of Things (IoT) devices. These devices often run lightweight operating systems, are left switched on for long periods, and are not always updated regularly. This makes them an attractive target for malware. One well-known example is the Mirai botnet, which infected large numbers of IoT devices and used them to carry out denial-of-service attacks [1]. Mirai and related families such as Gafgyt (also called Bashlite) are still used as reference malware families in IoT security research.

When such malware runs on a device it usually generates network traffic of its own, for example scanning other hosts, contacting a controller, or flooding a target. Because this behaviour is visible on the network, the traffic can be recorded as packet captures (PCAP files) and converted into numerical features such as header length, time-to-live, packet rate, flag counts and packet-size statistics. A machine-learning model can then be trained on these features to decide whether a piece of traffic looks normal or malicious.

Our project works with this idea. We use the CIC-YNU-IoTMal dataset, released in 2026 by the Canadian Institute for Cybersecurity together with Yokohama National University [2][3], and train deep neural networks on the network-traffic (PCAP) part of its x86 data.

1.2 Motivation

Our motivation came from three points. First, malware targeting IoT devices is a real and continuing security problem, and the network is one place where its behaviour can be observed without installing anything on the infected device. Second, the CIC-YNU-IoTMal dataset is recent (2026) and already provides processed network-traffic features with benign and malware labels, so it was possible to start experimenting with deep learning directly. Third, the Design Experience course expects us to carry a problem through the complete cycle of design, implementation, testing and evaluation, and this project let us do that on a topic related to our programme in Artificial Intelligence and Information Technology.

1.3 Problem Statement

Given a record of network-traffic features captured while a program was running in a sandbox, decide whether the traffic is benign or malicious, and, where possible, identify which malware family produced it. The task has to be solved using a deep learning model trained on labelled data, and the result has to be evaluated honestly on data the model has not seen during training.

1.4 Aim

To design, train and evaluate a deep learning model that detects malicious network traffic using the x86 PCAP data of the CIC-YNU-IoTMal dataset, and to present the trained model through a simple interactive detector.

1.5 Objectives

- To study and inspect the x86 PCAP portion of the CIC-YNU-IoTMal dataset (size, columns, data types and class distribution).
- To clean and prepare the data for training (unknown labels, unused columns, missing values, infinite values, train-test split and feature scaling).

- To build a neural network for five-class classification (Benign, DarkNexus, Gafgyt, Generic, Mirai) and evaluate it.
- To reformulate the problem as binary Benign-versus-Malicious classification based on what the first experiment showed, and to evaluate that model using accuracy, precision, recall, F1-score, a confusion matrix and ROC-AUC.
- To build an interactive Cyber Threat Detector in Google Colab using ipywidgets that classifies a selected sample as benign or malicious.
- To save the trained binary model so that it can be reused without retraining.
- To suggest possible future improvements.

1.6 Scope of the Project

The project covers the following:

- Only the x86 PCAP network-traffic file of the dataset. The ARM, MIPS and MIPSEL files and the SAR and STRACE data were not used.
- Two experiments with the same basic neural network: five-class classification and binary classification.
- Training, evaluation and a demonstration detector inside Google Colab.

The project does not cover live packet capture, real-time monitoring, deployment on a server or network device, or testing on traffic from outside the dataset. The detector is a demonstration that works on samples taken from our own test set. It is not a replacement for professional intrusion detection or security infrastructure.

1.7 Project Workflow

The work followed the steps shown in Figure 1.1. The dataset was loaded and inspected, preprocessed, and scaled. A deep learning model was then trained and evaluated, and finally the trained binary model was used inside an interactive detector.

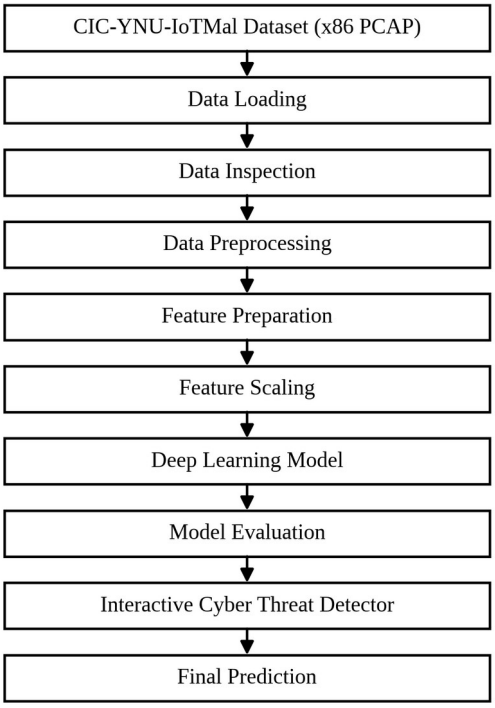


Figure 1.1: Overall Project Workflow

Figure 1.1 shows the sequence from the CIC-YNU-IoTMal dataset to the final prediction. The model-development block was run twice: first as a five-class problem and then as a binary problem (Chapters 8 and 9).

The actual development did not move in a straight line, and we have kept the report in the same order as the work. The first model we trained classified five categories and gave a test accuracy of only 32.72%. After studying the classification report and confusion matrix we changed the problem to binary classification, which gave a test accuracy of 98.02% and an ROC-AUC of 0.9980. The interactive detector and the model-saving step were built on the binary model.

1.8 Organization of the Report

Chapter 2 describes the system requirements and the technology used. Chapter 3 describes the dataset and Chapter 4 the preprocessing. Chapter 5 presents the exploratory analysis that we carried out. Chapter 6 explains the overall methodology and Chapter 7 the neural network design. Chapters 8 and 9 present Experiment 1 (five-class) and Experiment 2 (binary). Chapter 10 describes the interactive detector and model saving. Chapter 11 compares the results, Chapter 12 discusses them, and Chapter 13 gives the conclusion and future scope. References follow.

2. SYSTEM ANALYSIS AND PROJECT REQUIREMENTS

2.1 Existing Problem

Identifying malicious traffic from large volumes of network data is difficult to do by hand. Rules written for known malware can miss new variants, and fixed thresholds on individual values such as packet rate can produce many false alarms. IoT malware makes this harder because families such as Mirai and Gafgyt share much of their behaviour, so traffic from different families can look similar in the extracted features. A data-driven model that learns patterns from many labelled examples is one way to approach the problem, and this is the approach we studied.

2.2 Proposed System

The proposed system is a deep learning classifier trained on the processed network-traffic features of the CIC-YNU-IoTMal x86 PCAP file. The system takes a row of 39 numerical traffic features, scales it with the scaler learned from the training data, and passes it through a small fully connected neural network. For the final binary model the output is a probability that the traffic is malicious, which is converted into a BENIGN or MALICIOUS decision using a threshold of 0.5. An interactive widget in Colab lets a user choose a sample type and see the prediction and confidence.

2.3 System Objectives

- Classify network-traffic records as benign or malicious with high accuracy on unseen test data.
- Report the evaluation honestly, including the weak results of the first experiment.
- Provide a simple interface that demonstrates how the trained model makes a prediction.
- Keep the work reproducible by fixing the random seed of the train-test split (random_state = 42) and by saving the trained model.

2.4 Functional Requirements

Table 2.1: Functional Requirements

ID	Requirement	Implemented in the project as
FR1	Load the network-traffic dataset	pcap.parquet uploaded to Colab and read with pandas
FR2	Inspect the data	Shape, column names, first records, data types, label counts, missing values
FR3	Clean and prepare the data	Unknown labels removed, Hash and Arch dropped, missing and infinite values handled
FR4	Split and scale the data	80/20 stratified split; StandardScaler fitted on training data
FR5	Train a five-class model	Experiment 1: Dense 64, Dense 32, 5-neuron softmax output
FR6	Train a binary model	Experiment 2: same hidden layers, 1-neuron sigmoid output
FR7	Evaluate the models	Test accuracy, classification report, confusion matrix, ROC-AUC (binary)

ID	Requirement	Implemented in the project as
FR8	Predict for a selected sample	ipywidgets detector: dropdown, button and output area
FR9	Save the trained model	binary_model.save("cyber_threat_detector.keras")

2.5 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

Requirement	Description in this project
Accuracy	The binary model should detect malicious traffic reliably; it achieved 98.02% test accuracy and ROC-AUC 0.9980.
Performance	Training was fast: each epoch of 1,061 steps took about 3 to 7 seconds in the Colab runtime.
Reproducibility	Train-test split uses random_state = 42; the code is kept in one Colab notebook and the trained model is saved to a file.
Usability	The detector has one dropdown and one button; the result is printed as Prediction and Confidence.
Portability	Runs in a web browser on Google Colab without local installation.
Maintainability	Code is divided into small, commented notebook cells, each doing one step.

2.6 Hardware Requirements

Table 2.3: Hardware Requirements

Component	Purpose	Details
Computer / laptop	Access Google Colab through a web browser	**[TO BE FILLED FROM ACTUAL IMPLEMENTATION]**
Internet connection	Connect to Colab and upload the dataset file	Any stable connection
Cloud runtime (Colab)	Run data processing and training	Runtime type (CPU/GPU) and RAM: **[TO BE FILLED FROM ACTUAL IMPLEMENTATION]**
Storage	Hold the dataset file (pcap.parquet) and saved model	The CSV copy of the dataset is about 124 MB; the .keras file size: **[TO BE FILLED FROM ACTUAL IMPLEMENTATION]**

The model is small (4,673 trainable parameters for the binary model), so training did not need special hardware.

2.7 Software Requirements

- A modern web browser and a Google account to use Google Colab.
- Python 3 with the libraries listed in Table 2.4.
- The dataset file pcap.parquet (x86 PCAP data of CIC-YNU-IoTMal).

2.8 Technology Stack

Table 2.4: Software and Technology Stack

Tool / Library	Used for
Python 3	Programming language (the Colab traceback shows Python 3.13)
Google Colab	Notebook environment; file upload through google.colab.files
Pandas	Loading the parquet file, inspection, cleaning, label counts
NumPy	Numerical operations, np.isinf, np.where, np.argmax
Scikit-learn [6]	LabelEncoder, train_test_split, StandardScaler, compute_class_weight, classification_report, confusion_matrix, roc_curve, roc_auc_score
TensorFlow 2.21.0 [4]	Deep learning framework (version printed in the notebook)
Keras [5] (tf.keras)	Sequential model, Dense and Dropout layers, training and saving
Matplotlib	Class distribution chart, training curves, ROC curve
Seaborn	Heatmaps for confusion matrices
ipywidgets	Interactive detector: HTML title, Dropdown, Button, Output

2.9 System Inputs

For training, the input is the preprocessed table of 39 numerical features per record together with the label. For the detector, the input is a traffic-type selection (Benign Traffic or Malicious Traffic) made through the dropdown; the system then takes one record of that type from the test set and passes its 39 features to the model.

2.10 System Outputs

The outputs are the trained models, evaluation results (accuracy, classification report, confusion matrix, ROC curve and ROC-AUC), the saved file cyber_threat_detector.keras, and, for the detector, a printed Prediction (BENIGN or MALICIOUS) together with a Confidence percentage.

2.11 Development Environment

All work was done in a single Google Colab notebook. The dataset file was uploaded using the Colab upload widget and read into a pandas DataFrame. The notebook was run from top to bottom, with each step in its own cell. The code is in the Colab notebook submitted with this report.

3. DATASET DESCRIPTION

3.1 Dataset Overview

The project uses the CIC-YNU-IoTMal dataset, published in 2026 [2][3]. It is a recent IoT malware dataset and it is different from the older CIC IoT datasets. According to the official dataset page, it was prepared by the Canadian Institute for Cybersecurity (University of New Brunswick) together with Yokohama National University, and it combines static and dynamic analysis of IoT malware. Table 3.1 summarises the part of the dataset that we used.

Table 3.1: Dataset Summary

Item	Details
Dataset	CIC-YNU-IoTMal Dataset 2026
Source	Canadian Institute for Cybersecurity, University of New Brunswick, with Yokohama National University [3]
Architecture used	x86
Data used	PCAP (network traffic), loaded from pcap.parquet
Raw size in our notebook	455,641 records, 42 columns
After removing Unknown labels	424,110 records, 42 columns
After dropping Hash and Arch	424,110 records, 40 columns (39 features + MalwareFamily)
Categories used	Benign, DarkNexus, Gafgyt, Generic, Mirai

3.2 CIC-YNU-IoTMal Dataset

The official page describes the dataset as a multilayer collection for static and dynamic analysis of IoT malware behaviour [3]. About 10,000 malware samples taken from the IoTPOT dataset were executed in controlled virtual environments, and three kinds of behaviour were recorded: network traffic (PCAP), system calls (STRACE) and system activity (SAR). The data is organised by CPU architecture: ARM, MIPS, MIPSEL and x86. The malware families listed on the page include Mirai, Bashlite (Gafgyt), DarkNexus, Rudedevil, Agent, Generic and Tsunami, and the page states that the benign samples were generated using large language models.

The dynamic data was collected with an open-source sandbox developed by the authors, in which the samples run inside QEMU virtual machines with OpenWrt, and network traffic is captured with tshark [3]. The dataset is described in the publication by Dadkhah et al. [2].

3.3 Why This Dataset Was Selected

The dataset suited our project for the following reasons, which we checked against the official description:

- It is recent (2026) and was published by a well-known research group in cybersecurity.
- It contains both benign and malicious samples, with malware family labels, which allowed both a five-class and a binary experiment.
- The network-traffic data is already provided in a processed, tabular form, so it could be used directly with pandas and a neural network.
- It is large enough (hundreds of thousands of records) to train a neural network.

3.4 X86 PCAP Dataset

We used only the x86 PCAP file. Among the four architectures, the x86 PCAP file has the smallest number of records (455,641, compared with 737,651 for ARM, 870,017 for MIPS and 1,104,016 for MIPSEL according to the dataset page [3]), which is easier to handle in a Colab notebook. The official page lists 40 features for the PCAP files, while the file we loaded had 42 columns. The extra columns are the three descriptive columns Hash, MalwareFamily and Arch, and we did not investigate how the 40 on the page is counted. The notebook code that loads the file is shown in Figure 3.1.

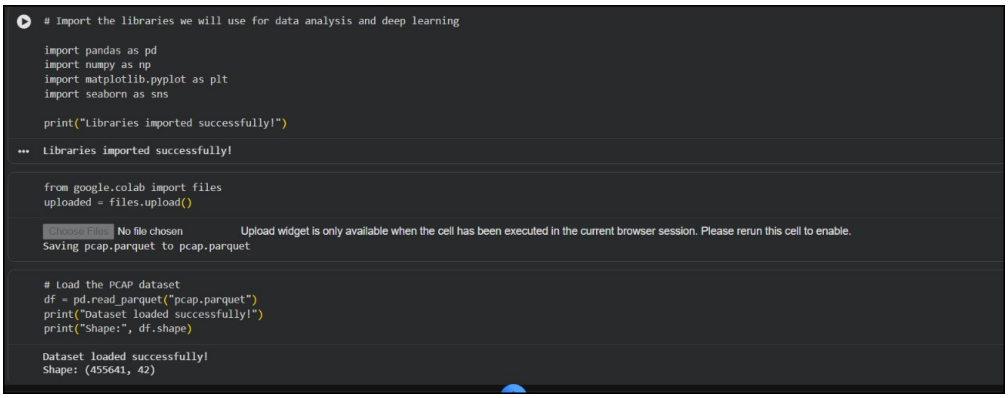


Figure 3.1: Dataset Loading and Dataset Shape in Google Colab

The notebook imports the libraries, uploads pcap.parquet, and prints the dataset shape (455641, 42).

3.5 Dataset Dimensions

After loading, the DataFrame had 455,641 rows and 42 columns. Removal of the Unknown label reduced it to 424,110 rows (Chapter 4). All 424,110 remaining rows have Arch = x86, as expected for this file.

3.6 Dataset Attributes

The column names printed by the notebook are shown in Figure 3.2, and Table 3.2 groups them. The column names are used as given in the dataset; for the exact meaning of each feature the dataset documentation should be consulted. We did not change the names.

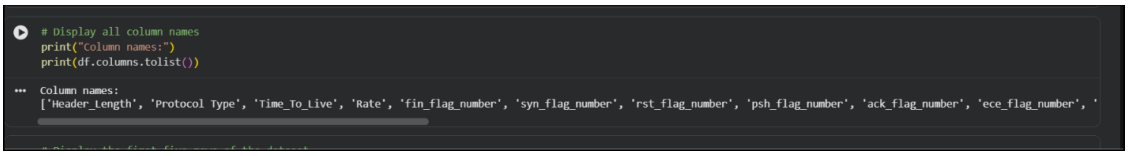


Figure 3.2: Column Names of the Dataset

The list is cut off by the notebook output width, so the complete set of columns is given in Table 3.2.

Table 3.2: Important Dataset Attributes

Group	Columns	Data type
Basic header features (4)	Header_Length, Protocol Type, Time_To_Live, Rate	float64 (Protocol Type: int64)
TCP flag values (7)	fin_flag_number, syn_flag_number, rst_flag_number, psh_flag_number, ack_flag_number, ece_flag_number, cwr_flag_number	float64
Flag counts (4)	ack_count, syn_count, fin_count, rst_count	int64

Group	Columns	Data type
Protocol indicators (15)	HTTP, HTTPS, DNS, Telnet, SMTP, SSH, IRC, TCP, UDP, DHCP, ARP, ICMP, IGMP, IPv, LLC	float64
Packet-size and timing statistics (9)	Tot sum, Min, Max, AVG, Std, Tot size, IAT, Number, Variance	int64: Tot sum, Min, Max, Number; float64: others
Descriptive columns (3)	Hash, MalwareFamily, Arch	object

The 39 model features are the first five groups (4 + 7 + 4 + 15 + 9). MalwareFamily is the target. Hash and Arch were removed before training (Section 4.5). The first five records of the dataset are shown in Table 3.3; the table shows a selection of columns only, as the notebook output also did.

Table 3.3: First Five Records of the Dataset (Selected Columns)

Header_Length	Protocol Type	Time_To_Live	Rate	Max	AVG	Std	IAT	Number	Variance	Malware Family
7.2	17	61.2	8.7066	342	168.4	120.1066	0.1149	10	14425.60	Benign
4.8	17	64	6.1822	342	103	85.1208	0.2132	10	7245.56	Benign
4.8	17	51.2	4.5535	342	176.4	143.5016	0.2196	10	20592.71	Benign
3.2	1	51.2	5.7938	342	98.6	87.8233	0.1726	10	7712.93	Benign
5.6	17	64	3.7606	342	155.8	129.0700	0.2695	10	16659.07	Benign

3.7 Target Variable

The target variable is the MalwareFamily column. In Experiment 1 it is used as five classes. In Experiment 2 it is converted to a binary label (Benign = 0, Malicious = 1).

3.8 Malware/Traffic Categories

After removing records with the Unknown label, five categories remained: Benign (normal traffic), and the malware families DarkNexus, Gafgyt, Generic and Mirai. The raw label counts, including Unknown, are shown in Figure 3.3.

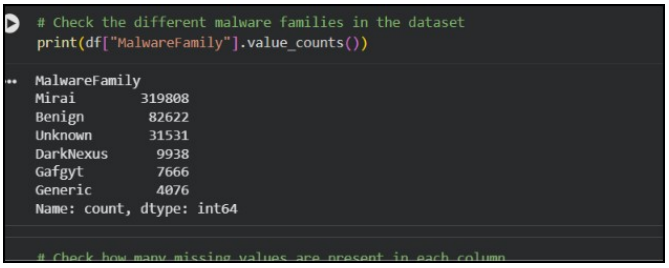


Figure 3.3: MalwareFamily Counts in the Raw Dataset

The raw file contains 31,531 records with the label Unknown, which were removed before training.

3.9 Class Distribution

Table 3.4: Class Distribution of the Dataset

Class	Records	Share of cleaned data
Mirai	319,808	75.4%
Benign	82,622	19.5%
DarkNexus	9,938	2.3%
Gafgyt	7,666	1.8%
Generic	4,076	1.0%
Total (cleaned)	424,110	100%
Unknown (removed)	31,531	not used

The percentages in Table 3.4 were calculated by us from the counts printed by the notebook. The dataset is clearly imbalanced: Mirai alone makes up about three quarters of the cleaned records, while Generic is only about 1%.

3.10 Dataset Characteristics

- The data is tabular: each record is one row of numerical traffic features, not a raw packet.
- The dataset is imbalanced in both the five-class and the binary view (about 19.5% benign and 80.5% malicious).
- Records come from malware and benign programs executed in a sandbox. Many rows can come from the same executed sample, which the Hash column identifies.
- The official page states that benign samples were generated using large language models [3]. This means the benign traffic is not traffic from real users, and we keep this in mind when interpreting our results.

4. DATA PREPROCESSING

This chapter describes, in the order in which they were done in the notebook, the steps used to turn the raw dataset into the arrays used for training. Only steps that are present in the notebook are described.

4.1 Data Loading

The dataset file pcap.parquet was uploaded into the Colab session using the upload widget (google.colab.files.upload) and read into a pandas DataFrame with pd.read_parquet. The shape printed after loading was (455641, 42) as shown earlier in Figure 3.1.

4.2 Initial Dataset Inspection

Before changing anything we inspected the data with simple pandas commands: the shape, the list of columns, the first five records, the count of each MalwareFamily label, the number of missing values in each column, and the data type of each column. This showed that 39 of the 42 columns were numeric (Hash, MalwareFamily and Arch are text columns), that the labels included an Unknown class, and that two columns contained missing values.

4.3 Data Cleaning

The cleaning steps were applied in this order: (1) remove records with the Unknown label, (2) drop the Hash and Arch columns, (3) fill missing values in Std and Variance, (4) split the data, (5) detect and replace infinite values, and (6) scale the features. Each step is explained below.

4.4 Handling Unknown Labels

The raw dataset contains 31,531 records whose MalwareFamily is Unknown (Figure 3.3). The dataset page also notes that Unknown labels are present in the data and can be dropped before applying machine-learning algorithms [3]. Since these records have no usable class, we removed them. The shape changed from (455641, 42) to (424110, 42), as shown in Figure 4.1.

4.5 Removal of Unnecessary Information

The Hash and Arch columns were dropped, which reduced the shape to (424110, 40). Hash is an identifier of the executed sample rather than a traffic measurement, and Arch only records the architecture, which is x86 for every record in this file. Neither is a traffic feature that the model should learn from.

```
# Remove rows where the malware family is unknown
df = df[df["MalwareFamily"] != "Unknown"]
print("Unknown labels removed.")
print("New shape:", df.shape)

Unknown labels removed.
New shape: (424110, 42)

# Remove columns that are not useful for our prediction model
df = df.drop(["Hash", "Arch"], axis=1)
print("Hash and Arch columns removed.")
print("New shape:", df.shape)

Hash and Arch columns removed.
New shape: (424110, 40)

# Fill missing values in numerical columns with the median value
df["Std"] = df["Std"].fillna(df["Std"].median())
df["Variance"] = df["Variance"].fillna(df["Variance"].median())
print("Missing values handled.")

Missing values handled.
```

Figure 4.1: Removal of Unknown Labels, Dropping Hash and Arch, and Median Filling

The three cleaning cells and their outputs. The final line of the median-filling cell prints Missing values handled.

4.6 Missing Value Analysis

The missing-value check showed that only two columns, Std and Variance, had missing values, 483 each in the raw file (Figure 4.2). The other columns had none. These were filled with the median of the respective column, using the following code:

```
df["Std"] = df["Std"].fillna(df["Std"].median())
df["Variance"] = df["Variance"].fillna(df["Variance"].median())
```

```
HTTP            0
HTTPS           0
DNS             0
Telnet          0
SMTP            0
SSH             0
IRC             0
TCP             0
UDP             0
DHCP            0
ARP             0
ICMP            0
IGMP            0
IPv             0
LLC             0
Tot sum         0
Min             0
Max             0
AVG             0
Std           483
Tot size        0
IAT             0
Number          0
Variance      483
Hash            0
MalwareFamily   0
Arch            0
dtype: int64
```

Figure 4.2: Missing Values per Column Before Cleaning

The lower part of the output shows 483 missing values in each of Std and Variance. All other columns show 0.

After this step the check printed 0 missing values and a shape of (424110, 40). We note that these two medians were calculated on the whole cleaned dataset, before it was split into training and test sets.

4.7 Infinite Value Analysis

Infinite values were discovered only when we tried to scale the features for the first time. StandardScaler stopped with a ValueError (Figure 4.5), because the data contained values it

could not handle. We therefore counted infinite values with np.isinf and found 378 in the training data and 87 in the testing data (Figure 4.3). In the training data, the only column that contained infinite values was Rate.

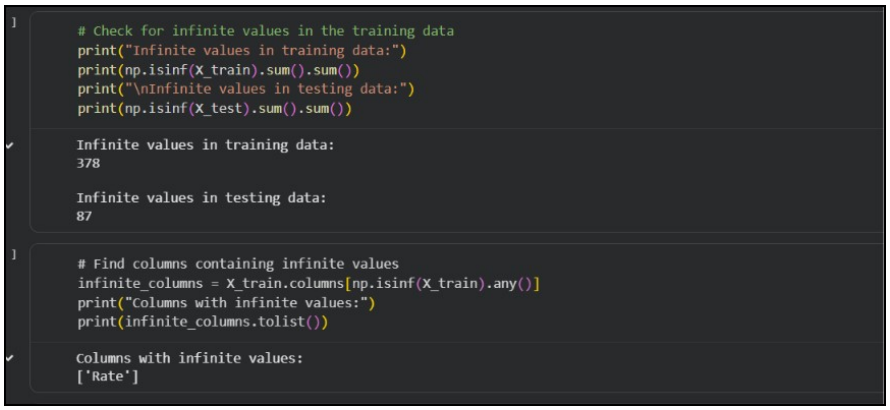


Figure 4.3: Detection of Infinite Values in the Training and Testing Data

378 infinite values in the training data and 87 in the testing data; the column found in the training data is Rate.

To fix this, the infinite values in both sets were replaced with NaN, and all NaN values were then filled with the column medians calculated from the training data only (train_medians = X_train.median()). Using the training medians for the test set avoids using test information in the preprocessing. The check after this step is shown in Figure 4.4.

```
Infinite values in training: 0
Infinite values in testing: 0
Missing values in training: 0
Missing values in testing: 0
```

Figure 4.4: Verification of Missing and Infinite Values After Data Preprocessing

Output of the verification cell in the notebook (transcribed from the notebook output). All four counts are zero, so the data is safe to scale.

4.8 Feature Preparation

The input features X were separated from the target by dropping the MalwareFamily column, which left 39 numerical columns and 424,110 rows. The target y is the MalwareFamily column of the same length. No categorical feature encoding was needed for X because all remaining columns are numeric.

4.9 Target Label Preparation

The class names were converted to integers with scikit-learn's LabelEncoder. The encoder orders the names alphabetically, which gave the mapping 0 = Benign, 1 = DarkNexus, 2 = Gafgyt, 3 = Generic and 4 = Mirai (Figure 4.5). The binary labels used in Experiment 2 were derived later from these encoded labels (Section 9.2).

4.10 Train-Test Split

The data was split with train_test_split using test_size = 0.2, random_state = 42 and stratify set to the encoded five-class label. Stratification keeps the proportion of each class the same in both sets. This gave 339,288 training records and 84,822 testing records. Figure 4.5 shows the code

and the shapes; the traceback below the shapes is the ValueError from the first scaling attempt mentioned above.

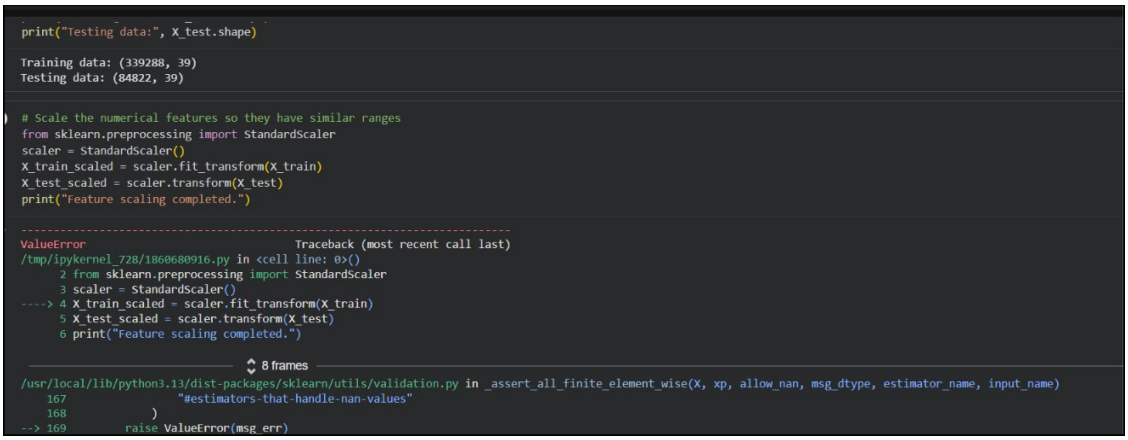


Figure 4.5: Train-Test Split Shapes and the Error Raised by the First Scaling Attempt

The split shapes (339288, 39) and (84822, 39) are printed first. The first StandardScaler call then failed with a ValueError, which led us to check for infinite values.

4.11 Feature Scaling

The 39 features have very different ranges. For example, Variance takes values in the thousands while flag values are 0 or 1. Neural networks usually train better when inputs are on similar scales, so StandardScaler was used. It converts each feature using z = (x - mean) / standard deviation. The scaler was fitted on the training data only (fit_transform) and then applied to the test data (transform), so that no information from the test set is used when computing the mean and standard deviation. The same fitted scaler is also used inside the detector in Chapter 10.

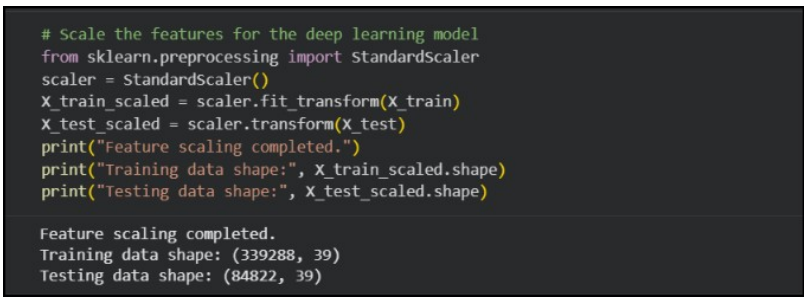


Figure 4.6: Feature Scaling and Final Training and Testing Shapes

After scaling, the training array has shape (339288, 39) and the testing array has shape (84822, 39).

4.12 Final Dataset Dimensions

The final scaled arrays have the following shapes: training (339288, 39) and testing (84822, 39). Both contain no missing and no infinite values, as verified in Figure 4.4.

4.13 Preprocessing Summary

Table 4.1: Preprocessing Summary

Step	Action	Result
1	Load pcap.parquet into pandas	455,641 rows, 42 columns
2	Remove records labelled Unknown	424,110 rows, 42 columns
3	Drop Hash and Arch	424,110 rows, 40 columns

Step	Action	Result
4	Fill missing Std and Variance with median	0 missing values
5	Separate features and target; encode labels	X: 39 features; 5 classes
6	Stratified 80/20 split, random_state = 42	339,288 train / 84,822 test
7	Replace infinite values (Rate) with NaN and fill with training medians	378 train and 87 test infinite values removed
8	StandardScaler fitted on training data	(339288, 39) and (84822, 39)

5. EXPLORATORY DATA ANALYSIS

5.1 Purpose of EDA

The purpose of the exploratory analysis was to understand the structure of the dataset and to find problems that must be handled before training. Our analysis was simple and is limited to what is described in this chapter. We did not carry out correlation analysis, feature histograms or feature-importance analysis in the notebook.

5.2 Class Distribution

The number of records of each malware family was counted with value_counts and plotted as a bar chart (Figure 5.1). The counts are the same as in Table 3.4.

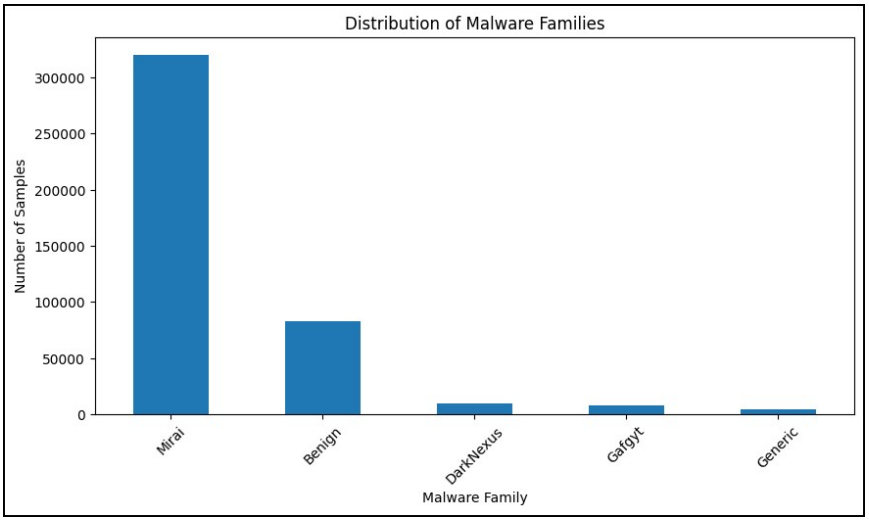


Figure 5.1: Distribution of Malware Families After Cleaning

Mirai has about 320,000 records and Benign about 83,000, while DarkNexus, Gafgyt and Generic each have fewer than 10,000.

5.3 Feature Analysis

The feature-level inspection consisted of listing the 42 column names, viewing the first five records (Table 3.3), checking the data types (Table 3.2), counting missing values per column and counting infinite values. These checks led to the discovery of the missing values in Std and Variance and the infinite values in Rate.

5.4 Observations

- The dataset is strongly imbalanced: Mirai is 75.4% of the cleaned records and Generic is only 1.0%.
- Most features are numeric, so no encoding of input features was required.
- Only Std and Variance had missing values (483 each in the raw file).
- The Rate column contained infinite values, which stopped the scaler until they were handled.
- The raw data contained an Unknown class (31,531 records) that cannot be used for supervised training.

5.5 Impact of Class Imbalance

Class imbalance affects both experiments. In the five-class problem, a model can reach a reasonable accuracy by favouring the large class (Mirai), while the small classes are almost ignored. In the binary problem, malicious records are about 80.5% of the test set, so a model that labels everything as malicious would already have an accuracy of about 80.5% (calculated by us from the test counts 68,298 out of 84,822). Therefore accuracy alone is not enough, and we also report precision, recall, F1-score, the confusion matrix and ROC-AUC. To reduce the effect of imbalance during training we used balanced class weights in both experiments (Sections 8.3 and 9.4).

6. PROPOSED METHODOLOGY

6.1 Overall Methodology

The methodology has four stages: data preparation, model development, evaluation, and demonstration. In the model-development stage the same small neural network was used for two different problem formulations, which allowed a direct comparison between them.

6.2 System Workflow

Figure 6.1 shows the layers of the proposed system, from the dataset to the final benign or malicious result.

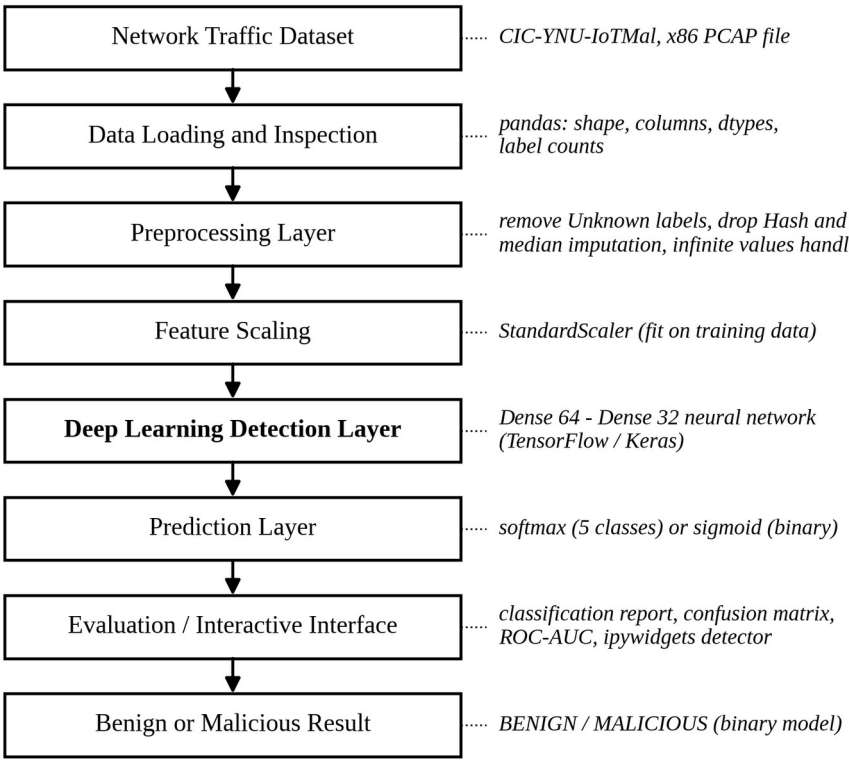


Figure 6.1: Proposed System Architecture

Each box is one layer of the system. The text on the right names what was used in that layer in our notebook.

6.3 Data Processing Pipeline

The pipeline consists of the steps described in Chapter 4: loading, removal of Unknown labels, dropping Hash and Arch, median filling of Std and Variance, label encoding, stratified splitting, handling of infinite values in Rate, and scaling with StandardScaler. The same prepared training and test arrays were used for both experiments.

6.4 Multiclass Classification Approach

In Experiment 1 the model predicts one of five classes (Benign, DarkNexus, Gafgyt, Generic, Mirai). The output layer has five neurons with a softmax activation, the loss is sparse categorical crossentropy and balanced class weights are used. This experiment tests whether the traffic features are enough to separate individual malware families.

6.5 Binary Classification Approach

In Experiment 2 the problem was reformulated. Benign was kept as 0 and all four malware families were combined into 1 (Malicious). The output layer has one neuron with a sigmoid activation, the loss is binary crossentropy, and balanced class weights are again used. The model outputs the probability that a record is malicious.

6.6 Model Training

Both models were trained with TensorFlow/Keras using the Adam optimizer for 10 epochs with a batch size of 256 and a validation split of 0.2. The details are given in Chapter 7.

6.7 Model Evaluation

The models were evaluated on the 84,822 test records, which were not used for training or validation. The following measures were used:

- Accuracy: the fraction of test records that are classified correctly.
- Precision: of the records predicted as a class, the fraction that really belong to it.
- Recall: of the records that really belong to a class, the fraction that the model finds.
- F1-score: the harmonic mean of precision and recall.
- Confusion matrix: a table of actual classes against predicted classes.
- ROC-AUC (binary model): the area under the curve of true positive rate against false positive rate [8]. A value of 1.0 means perfect ranking of malicious above benign records and 0.5 means random ranking.

6.8 Interactive Detection

The binary model, the fitted scaler and a threshold of 0.5 are used inside an ipywidgets interface. The user selects a traffic type, a sample of that type is taken from the test set, scaled and passed to the model, and the predicted class and confidence are displayed (Chapter 10).

7. DEEP LEARNING MODEL DEVELOPMENT

7.1 Neural Network Architecture

Both experiments use a fully connected (dense) feed-forward network with two hidden layers. The hidden-layer structure is the same in the two experiments; only the output layer is different (Figure 7.1).

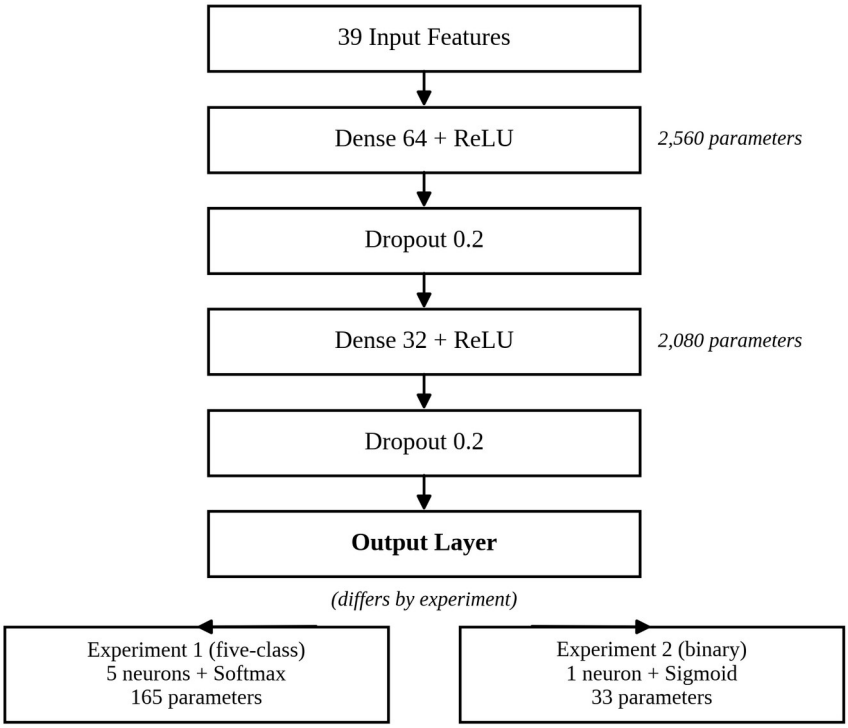


Figure 7.1: Deep Learning Model Architecture

The same hidden layers were used for both experiments. The output layer has 5 neurons with softmax for the five-class model and 1 neuron with sigmoid for the binary model. Parameter counts are taken from the model summaries.

7.2 Input Layer

The input layer takes the 39 scaled features of one network-traffic record, defined in Keras as Input(shape=(39,)).

7.3 Hidden Layers

The first hidden layer is a dense layer of 64 neurons and the second a dense layer of 32 neurons. Both use the ReLU activation. The first layer has 39 x 64 + 64 = 2,560 parameters and the second 64 x 32 + 32 = 2,080 parameters, matching the model summaries.

7.4 Dropout Layers

A dropout layer with rate 0.2 follows each hidden layer. During training, dropout randomly switches off 20% of the neurons in each update, which reduces the chance that the network depends too much on a few neurons . Dropout layers have no trainable parameters and are not active during testing and prediction.

7.5 Output Layer

For Experiment 1 the output layer is Dense(5, softmax), with 32 x 5 + 5 = 165 parameters. For Experiment 2 it is Dense(1, sigmoid), with 32 + 1 = 33 parameters.

7.6 Activation Functions

ReLU (rectified linear unit) outputs zero for negative inputs and the input itself for positive inputs. Softmax converts the five outputs into probabilities that add up to one, and the largest probability gives the predicted family. Sigmoid maps a single output to a value between 0 and 1, which is read as the probability of the Malicious class.

7.7 Loss Functions

Sparse categorical crossentropy was used for Experiment 1, since the labels are integers from 0 to 4. Binary crossentropy was used for Experiment 2, since the labels are 0 or 1.

7.8 Optimizer

Both models used the Adam optimizer [7] with the default Keras settings (optimizer="adam"). We did not tune the learning rate or other optimizer settings.

7.9 Training Parameters

Table 7.1: Training Parameters

Parameter	Experiment 1 (five-class)	Experiment 2 (binary)
Framework	TensorFlow 2.21.0 / Keras	TensorFlow 2.21.0 / Keras
Optimizer	Adam	Adam
Loss	Sparse categorical crossentropy	Binary crossentropy
Metric	Accuracy	Accuracy
Epochs	10	10
Batch size	256	256
Validation split	0.2	0.2
Class weights	Used (balanced, 5 classes)	Used (balanced, 2 classes)
Training records	339,288	339,288
Steps per epoch	1,061	1,061
Time per epoch	about 3 to 7 s	about 3 to 6 s

7.10 Model Summary

Table 7.2 lists the layers of the two models, and the model summaries printed by Keras are shown in Figure 7.2 and Figure 7.3. The five-class model has 4,805 trainable parameters and the binary model has 4,673. Both are small networks.

Table 7.2: Model Architecture

Layer	Output shape	Activation	Parameters (Exp. 1)	Parameters (Exp. 2)
Input	(None, 39)	-	0	0
Dense	(None, 64)	ReLU	2,560	2,560

Layer	Output shape	Activation	Parameters (Exp. 1)	Parameters (Exp. 2)
Dropout (0.2)	(None, 64)	-	0	0
Dense	(None, 32)	ReLU	2,080	2,080
Dropout (0.2)	(None, 32)	-	0	0
Output Dense	(None, 5) / (None, 1)	Softmax / Sigmoid	165	33
Total			4,805	4,673

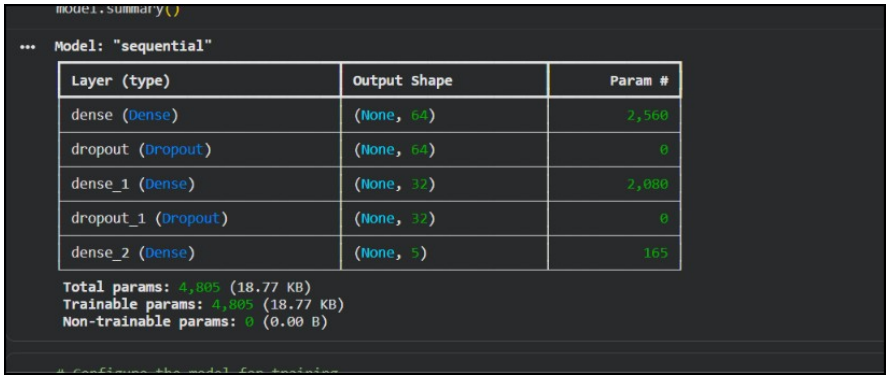


Figure 7.2: Model Summary of the Five-Class Model (Experiment 1)

Total parameters: 4,805, all trainable.

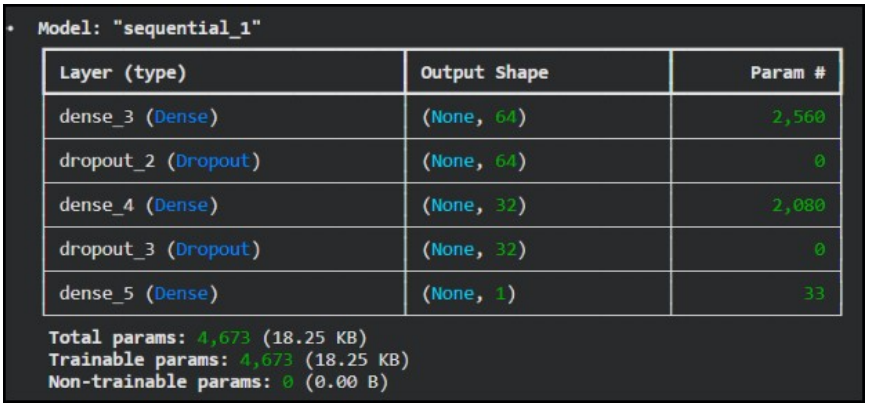


Figure 7.3: Model Summary of the Binary Model (Experiment 2)

Total parameters: 4,673, all trainable. The image is cropped to the summary table.

7.11 Training Performance

The training behaviour of the two models was very different. The five-class model stayed at a training accuracy of about 37% to 39% for all ten epochs, whereas the binary model reached about 95.8% accuracy in the first epoch and about 97.8% by the last. The epoch-by-epoch figures are given in Table 8.2 and Table 9.2, and the curves are shown in Figure 8.1, Figure 8.2 and Figure 9.4.

8. EXPERIMENT 1 — MALWARE FAMILY CLASSIFICATION

8.1 Experiment Objective

The objective of the first experiment was to check whether the 39 network-traffic features are enough to tell the individual malware families apart. The model had to assign each test record to one of five classes: Benign, DarkNexus, Gafgyt, Generic or Mirai.

8.2 Dataset Preparation

The prepared and scaled arrays described in Chapter 4 were used, with the five-class integer labels (0 = Benign, 1 = DarkNexus, 2 = Gafgyt, 3 = Generic, 4 = Mirai). Table 8.1 shows the number of records of each class in the test set (taken from the classification report) and in the training set. The training counts were obtained by us by subtracting the test counts from the class totals in Table 3.4, and they add up to 339,288.

Table 8.1: Class Distribution and Class Weights in Experiment 1

Class (label)	Training records	Test records	Class weight
Benign (0)	66,098	16,524	1.0266
DarkNexus (1)	7,950	1,988	8.5355
Gafgyt (2)	6,133	1,533	11.0643
Generic (3)	3,261	815	20.8088
Mirai (4)	255,846	63,962	0.2652
Total	339,288	84,822	

8.3 Model Configuration

The network is the one described in Chapter 7: 39 inputs, Dense 64 (ReLU), Dropout 0.2, Dense 32 (ReLU), Dropout 0.2, and an output layer of 5 neurons with softmax. It was compiled with the Adam optimizer, sparse categorical crossentropy loss and accuracy as the metric. Because the classes are very unequal in size, class weights were calculated with compute_class_weight(class_weight="balanced"). The weights in Table 8.1 show that the rarest class, Generic, was given about 20 times the weight of a Benign record and about 78 times the weight of a Mirai record.

8.4 Training Process

The model was trained with model.fit on the scaled training data for 10 epochs with a batch size of 256, a validation split of 0.2 and the class weights above. Each epoch had 1,061 steps.

8.5 Training and Validation Results

The results of the ten epochs are given in Table 8.2. Training accuracy stayed between 36.98% and 38.64%, and it was 37.12% in epoch 10. Validation accuracy changed from epoch to epoch between 29.47% and 38.68%, with the best value of 38.68% in epoch 2 and 32.54% in epoch 10. The training loss decreased steadily from 1.0010 to 0.7432, but the validation loss stayed close to 1.0 and did not improve (Figure 8.1 and Figure 8.2).

Table 8.2: Epoch-wise Training Results of Experiment 1

Epoch	Training accuracy	Training loss	Validation accuracy	Validation loss
1	0.3730	1.0010	0.2947	1.0120
2	0.3698	0.8552	0.3868	0.9299
3	0.3853	0.8114	0.3263	0.9894
4	0.3864	0.7927	0.3358	0.9687
5	0.3832	0.7798	0.3400	0.9502
6	0.3798	0.7705	0.3475	0.9946
7	0.3767	0.7588	0.3450	0.9627
8	0.3737	0.7524	0.3581	0.9578
9	0.3726	0.7466	0.3374	0.9483
10	0.3712	0.7432	0.3254	0.9981

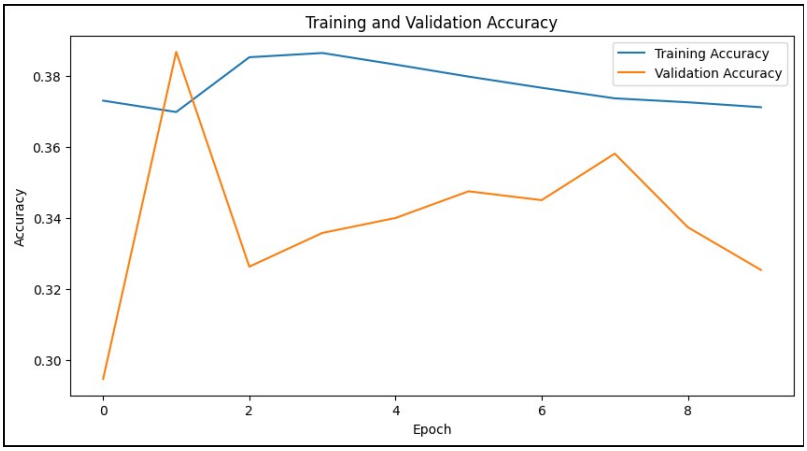


Figure 8.1: Training and Validation Accuracy of the Five-Class Model

Training accuracy stays near 0.37 to 0.39, and validation accuracy fluctuates between about 0.29 and 0.39.

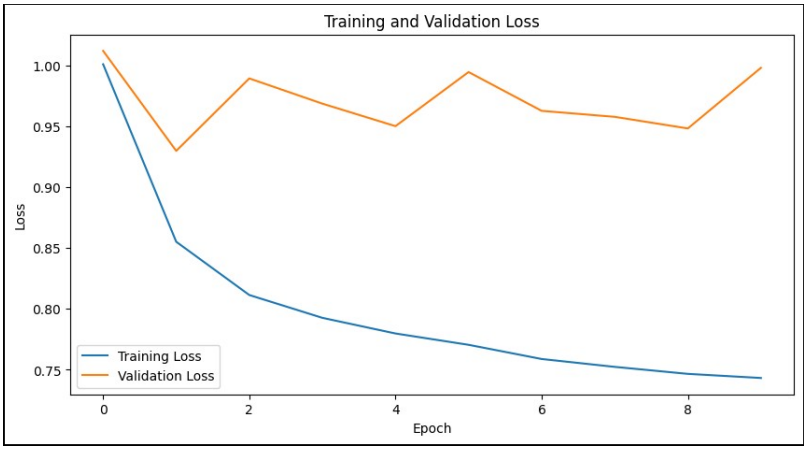


Figure 8.2: Training and Validation Loss of the Five-Class Model

Training loss falls from about 1.0 to 0.74 while validation loss remains close to 1.0.

On the test set the model reached a test accuracy of 32.72% (0.3272) and a test loss of 0.9994 (Figure 8.3).

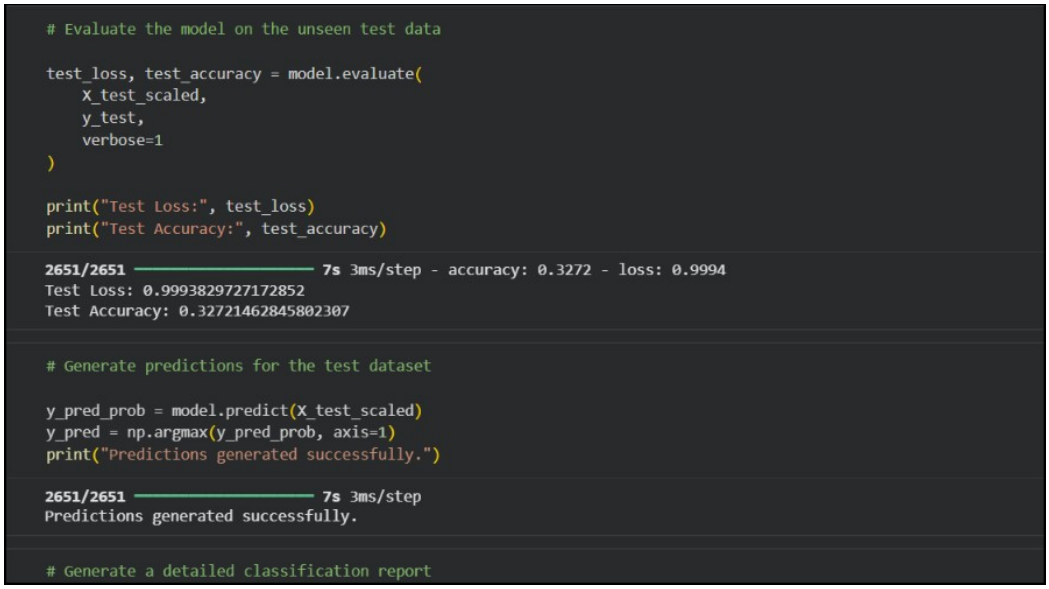


Figure 8.3: Test Evaluation and Prediction Generation for the Five-Class Model

model.evaluate on the test data printed accuracy 0.3272 and loss 0.9994.

8.6 Classification Report

The classification report in Figure 8.4 is summarised in Table 8.3.

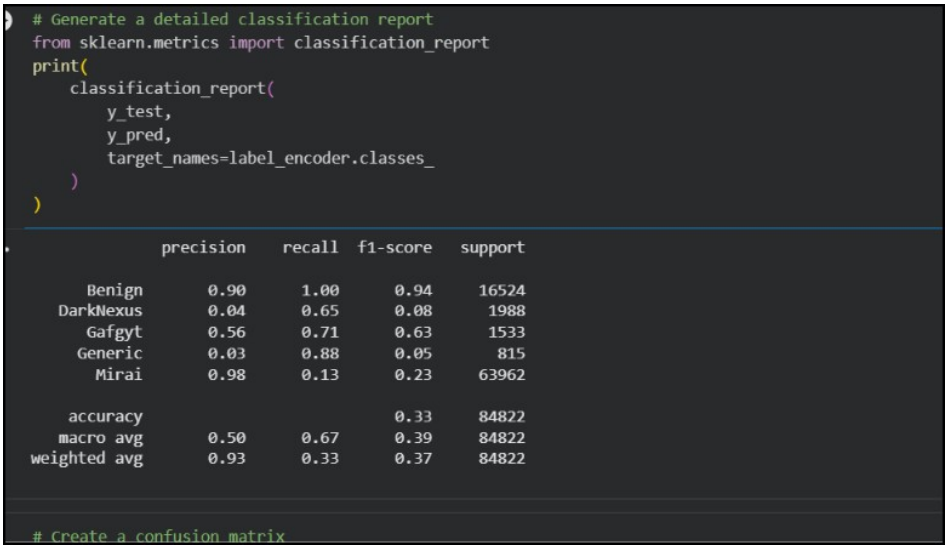


Figure 8.4: Classification Report of the Five-Class Model

Precision, recall, F1-score and support for each class. Overall accuracy is 0.33.

Table 8.3: Multiclass Classification Results (Test Set)

Class	Precision	Recall	F1-score	Support
Benign	0.90	1.00	0.94	16,524
DarkNexus	0.04	0.65	0.08	1,988
Gafgyt	0.56	0.71	0.63	1,533
Generic	0.03	0.88	0.05	815
Mirai	0.98	0.13	0.23	63,962
Accuracy			0.33	84,822

Class	Precision	Recall	F1-score	Support
Macro average	0.50	0.67	0.39	84,822
Weighted average	0.93	0.33	0.37	84,822

8.7 Confusion Matrix

The confusion matrix of the five-class model is shown in Figure 8.5 and its values are listed in Table 8.4. Each row is the actual class and each column the predicted class.

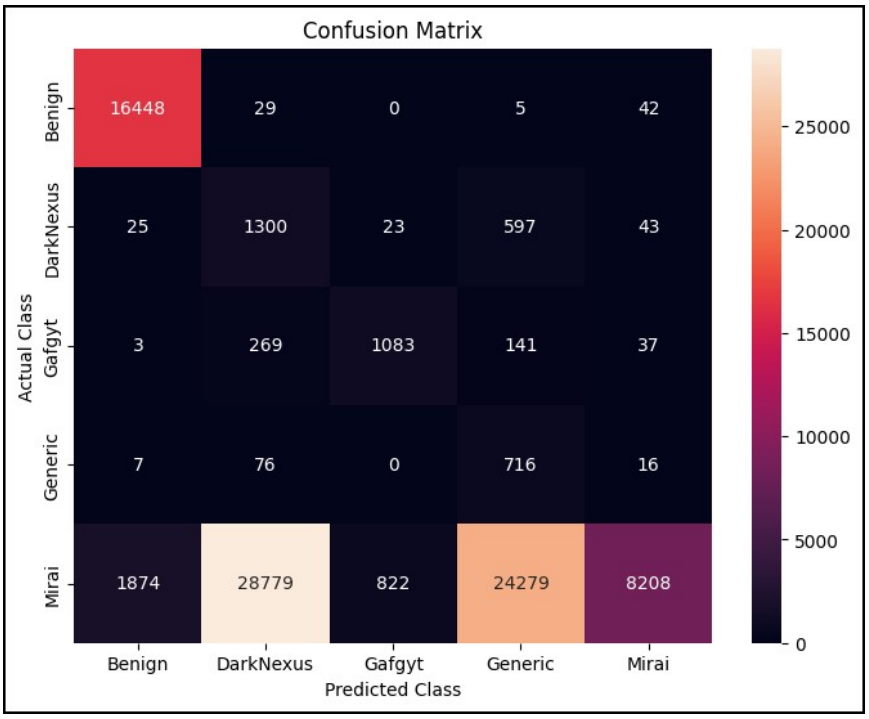


Figure 8.5: Confusion Matrix of the Five-Class Model

Most of the large Mirai row falls into the DarkNexus and Generic columns.

Table 8.4: Confusion Matrix Values of the Five-Class Model

Actual \ Predicted	Benign	DarkNexus	Gafgyt	Generic	Mirai
Benign	16,448	29	0	5	42
DarkNexus	25	1,300	23	597	43
Gafgyt	3	269	1,083	141	37
Generic	7	76	0	716	16
Mirai	1,874	28,779	822	24,279	8,208

8.8 Performance Analysis

The five-class model did not perform well. Its test accuracy of 32.72% is far below what we expected, and the F1-scores for DarkNexus (0.08), Generic (0.05) and Mirai (0.23) are low. The pattern in the confusion matrix is clear:

- Benign was recognised well. 16,448 of the 16,524 benign test records (recall 1.00 when rounded) were classified correctly.

- Mirai, the largest class, had a recall of only 0.13. Of its 63,962 test records, 28,779 were predicted as DarkNexus, 24,279 as Generic and 1,874 as Benign, and only 8,208 were predicted correctly.
- Because so many Mirai records were pushed into DarkNexus and Generic, the precision of these two classes fell to 0.04 and 0.03, even though their recall was 0.65 and 0.88.
- Gafgyt gave a more balanced result (precision 0.56, recall 0.71, F1-score 0.63).

The weighted average recall (0.33) is always equal to the accuracy, and here it is low because Mirai makes up about 75% of the test records and has a recall of only 0.13. The high weighted precision (0.93) should not be read as a good result for the same reason: Mirai has a precision of 0.98 because few records were predicted as Mirai and most of those were correct, and this one value dominates the weighted average.

We also looked at what happens if the five-class predictions are only read as benign or malware. Using the numbers in Table 8.4, we calculated that 82,837 of the 84,822 test records (97.66%) would be correct under that reading. This calculation was done by us from the confusion matrix for interpretation only; the model was not trained for that task. It suggests that the model could already separate benign traffic from malware traffic fairly well, and that the difficulty was in separating the families from each other.

8.9 Challenges Observed

- Class imbalance. The training set contains 255,846 Mirai records but only 3,261 Generic records. We used balanced class weights, but the weights are large for the small classes (up to 20.81 for Generic), and in our results the model seems to have favoured the small classes over Mirai.
- Overlapping traffic features. The families DarkNexus, Gafgyt, Generic and Mirai are all malware and produce similar kinds of traffic. The 39 features describe packets in summary form (header values, flags, sizes, timing), and a small network may not be able to separate the families from such features.
- Few distinct samples in the small classes. As an additional check done while writing this report (not part of the original notebook), we counted the distinct Hash values, which identify the executed samples. The cleaned data has 4,301 distinct samples, but only 50 for DarkNexus, 12 for Gafgyt and 12 for Generic, against 1,657 for Mirai and 2,570 for Benign. The small classes are therefore built from very few samples, each contributing many rows. This may make them harder to learn in a way that generalises.
- Limited training. The model was trained for only 10 epochs with default settings and no tuning. We cannot say from our experiments how much of the poor result is due to the data and how much to the simple setup.

8.10 Decision to Reformulate the Problem

The first experiment showed that distinguishing individual malware families was difficult with the selected features and with our network. Our main objective, however, is to detect whether traffic is malicious. Benign traffic had been separated well even in the five-class model, and the confusion between the malware families does not matter if the question is only whether the traffic is malicious. Based on this observation, we decided to reformulate the problem as binary classification, with Benign as 0 and all malware families together as 1. We kept the five-class experiment in the report because it explains why the final system was designed this way.

9. EXPERIMENT 2 — BINARY CYBER THREAT DETECTION

9.1 Experiment Objective

The objective of the second experiment was to train a model that decides whether a network-traffic record is benign or malicious. The workflow is shown in Figure 9.1.

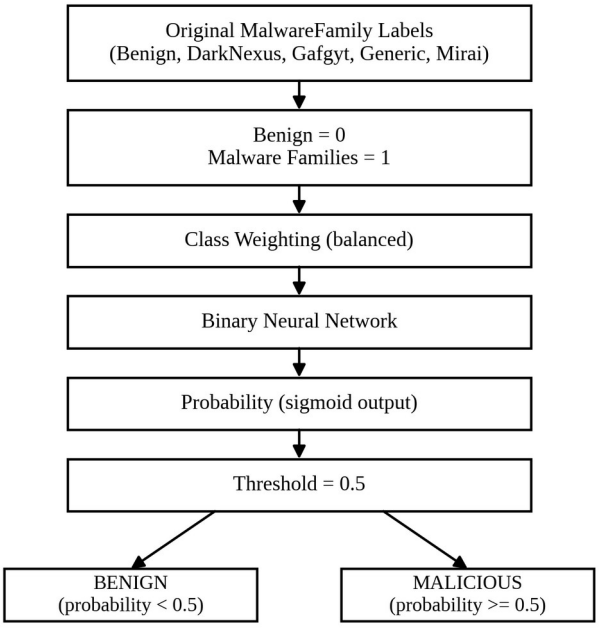


Figure 9.1: Binary Classification Workflow

The five-class labels are converted to binary labels, class weights are applied during training, and the output probability is compared with a threshold of 0.5.

9.2 Binary Label Conversion

The binary labels were created from the encoded five-class labels with the following code:

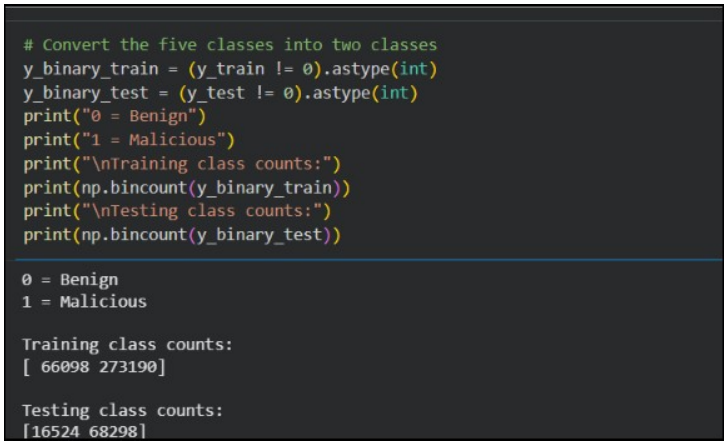
```
y_binary_train = (y_train != 0).astype(int)
y_binary_test = (y_test != 0).astype(int)
```

Since label 0 is Benign, all records with label 1, 2, 3 or 4 (DarkNexus, Gafgyt, Generic, Mirai) become 1 (Malicious). The same train-test split as in Experiment 1 was used, so the two experiments are evaluated on the same 84,822 test records.

9.3 Class Distribution

Table 9.1: Class Distribution in the Binary Experiment

Class	Training records	Test records
Benign (0)	66,098	16,524
Malicious (1)	273,190	68,298
Total	339,288	84,822



```
# Convert the five classes into two classes
y_binary_train = (y_train != 0).astype(int)
y_binary_test = (y_test != 0).astype(int)
print("0 = Benign")
print("1 = Malicious")
print("\nTraining class counts:")
print(np.bincount(y_binary_train))
print("\nTesting class counts:")
print(np.bincount(y_binary_test))
```

```
0 = Benign
1 = Malicious

Training class counts:
[ 66098 273190]

Testing class counts:
[16524 68298]
```

Figure 9.2: Binary Label Conversion and Class Counts

The printed counts match Table 9.1: [66098 273190] for training and [16524 68298] for testing.

Malicious records are about 80.5% of the training data (273,190 of 339,288) and the same share of the test data. The binary problem is therefore still imbalanced, but much less so than the five-class problem.

9.4 Class Weighting

Balanced class weights were calculated again for the two classes. The notebook printed 2.5666 for Benign and 0.6210 for Malicious (about 2.57 and 0.62). A benign record therefore counts about four times as much as a malicious record during training.

9.5 Model Architecture

The binary model uses the same hidden layers as Experiment 1 (Dense 64 with ReLU, Dropout 0.2, Dense 32 with ReLU, Dropout 0.2), followed by a single output neuron with a sigmoid activation (see Figure 7.1 and Table 7.2). It has 4,673 trainable parameters.

9.6 Training Configuration

The model was compiled with the Adam optimizer, binary crossentropy loss and accuracy as the metric, and trained for 10 epochs with a batch size of 256, a validation split of 0.2 and the binary class weights (see Table 7.1).

9.7 Training Performance

Table 9.2 lists the results of all ten epochs, as printed by the notebook (Figure 9.3). The notebook did not plot the training history of the binary model, so Figure 9.4 was drawn afterwards by us from the values in the epoch log, using Matplotlib.

```
Epoch 1/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.9579 - loss: 0.0955 - val_accuracy: 0.9752 - val_loss: 0.0621
Epoch 2/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9750 - loss: 0.0555 - val_accuracy: 0.9743 - val_loss: 0.0679
Epoch 3/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9763 - loss: 0.0525 - val_accuracy: 0.9789 - val_loss: 0.0571
Epoch 4/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9767 - loss: 0.0514 - val_accuracy: 0.9783 - val_loss: 0.0581
Epoch 5/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9774 - loss: 0.0491 - val_accuracy: 0.9775 - val_loss: 0.0564
Epoch 6/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9775 - loss: 0.0488 - val_accuracy: 0.9786 - val_loss: 0.0580
Epoch 7/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9779 - loss: 0.0483 - val_accuracy: 0.9778 - val_loss: 0.0596
Epoch 8/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9779 - loss: 0.0479 - val_accuracy: 0.9802 - val_loss: 0.0524
Epoch 9/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9776 - loss: 0.0475 - val_accuracy: 0.9800 - val_loss: 0.0523
Epoch 10/10
1061/1061 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9782 - loss: 0.0471 - val_accuracy: 0.9796 - val_loss: 0.0546
```

Figure 9.3: Training Log of the Binary Model

Epoch-wise accuracy, loss, validation accuracy and validation loss printed by model.fit.

Table 9.2: Epoch-wise Training Results of Experiment 2

Epoch	Training accuracy	Training loss	Validation accuracy	Validation loss
1	0.9579	0.0955	0.9752	0.0621
2	0.9750	0.0555	0.9743	0.0679
3	0.9763	0.0525	0.9789	0.0571
4	0.9767	0.0514	0.9783	0.0581
5	0.9774	0.0491	0.9775	0.0564
6	0.9775	0.0488	0.9786	0.0580
7	0.9779	0.0483	0.9778	0.0596
8	0.9779	0.0479	0.9802	0.0524
9	0.9776	0.0475	0.9800	0.0523
10	0.9782	0.0471	0.9796	0.0546

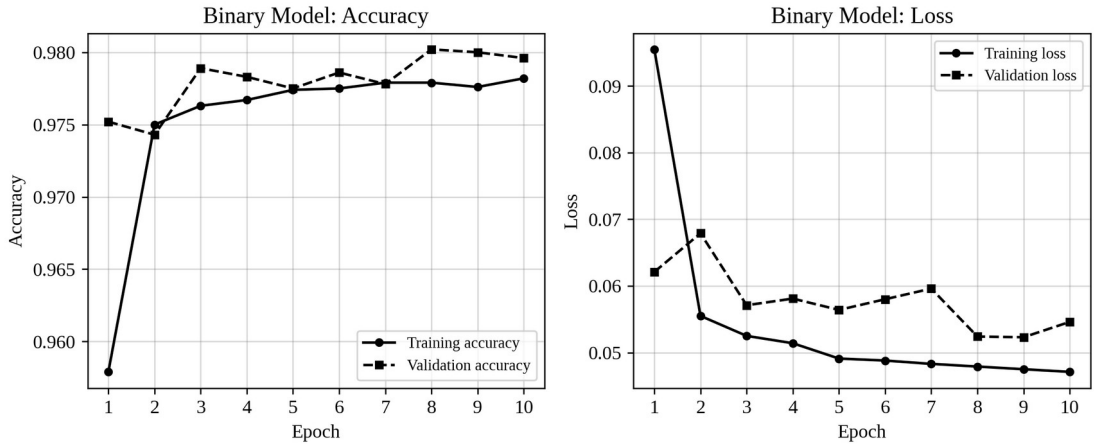


Figure 9.4: Training and Validation Accuracy and Loss of the Binary Model

Redrawn from the epoch log. Training accuracy rises from 0.9579 to 0.9782 and validation accuracy stays between 0.9743 and 0.9802. Training loss falls from 0.0955 to 0.0471.

The binary model learned much faster than the five-class model. Validation accuracy was already 97.52% after the first epoch and stayed between 97.43% and 98.02% afterwards, with

the highest value in epoch 8. Training and validation accuracy are close to each other, and we did not see a sign of strong overfitting in ten epochs.

9.8 Classification Report

The model was evaluated on the 84,822 test records. A threshold of 0.5 was used to turn the predicted probability into a class. The test loss was 0.0551 and the test accuracy was 0.9802 (98.02%) (Figure 9.5). The classification report is shown in Figure 9.6 and Table 9.3.

```
# Evaluate the binary model on unseen test data

binary_test_loss, binary_test_accuracy = binary_model.evaluate(
    X_test_scaled,
    y_binary_test,
    verbose=1
)
print("Test Loss:", binary_test_loss)
print("Test Accuracy:", binary_test_accuracy)

2651/2651 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.9802 - loss: 0.0551
Test Loss: 0.055061377584934235
Test Accuracy: 0.9802055954933167

# Generate predictions for the test dataset
binary_pred_prob = binary_model.predict(X_test_scaled)
binary_pred = (binary_pred_prob >= 0.5).astype(int).flatten()
print("Predictions generated successfully.")

2651/2651 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step
Predictions generated successfully.
```

Figure 9.5: Test Evaluation of the Binary Model

model.evaluate printed accuracy 0.9802 and loss 0.0551; predictions were generated with a 0.5 threshold.

```
# Generate the classification report
from sklearn.metrics import classification_report
print(
    classification_report(
        y_binary_test,
        binary_pred,
        target_names=["Benign", "Malicious"]
    )
)

              precision    recall  f1-score   support

      Benign       0.91      0.99      0.95     16524
   Malicious       1.00      0.98      0.99     68298

    accuracy                           0.98     84822
   macro avg       0.96      0.99      0.97     84822
weighted avg       0.98      0.98      0.98     84822
```

Figure 9.6: Classification Report of the Binary Model

Precision, recall and F1-score for Benign and Malicious on 84,822 test records.

Table 9.3: Performance of the Binary Classification Model (Test Set)

Class	Precision	Recall	F1-score	Support
Benign	0.91	0.99	0.95	16,524
Malicious	1.00	0.98	0.99	68,298
Accuracy			0.98	84,822
Macro average	0.96	0.99	0.97	84,822
Weighted average	0.98	0.98	0.98	84,822

Malicious precision is 1.00, which means that almost every record predicted as malicious really was malicious. Malicious recall is 0.98, so the model found about 98% of the malicious records. For Benign, recall is 0.99 but precision is lower at 0.91, which means that some records predicted as benign were actually malicious. This is explained by the confusion matrix.

9.9 Confusion Matrix

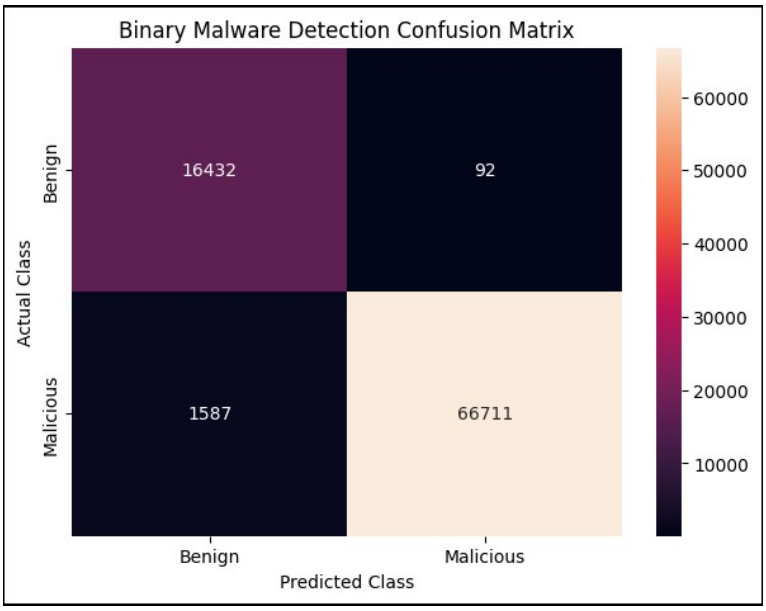


Figure 9.7: Confusion Matrix of the Binary Model

16,432 benign and 66,711 malicious records were classified correctly.

Table 9.4: Binary Confusion Matrix

Actual \ Predicted	Benign	Malicious	Total
Benign	16,432	92	16,524
Malicious	1,587	66,711	68,298
Total	18,019	66,803	84,822

In simple terms: 16,432 benign records were correctly called benign (true negatives) and 66,711 malicious records were correctly detected (true positives). 92 benign records were wrongly flagged as malicious (false positives), which is about 0.56% of the benign test records. 1,587 malicious records were missed and called benign (false negatives), which is about 2.32% of the malicious test records. These percentages were calculated by us from the matrix. In a security setting the missed records are the more serious error, because they are threats that were not detected. The accuracy of 98.02% is (16,432 + 66,711) / 84,822.

9.10 ROC Curve

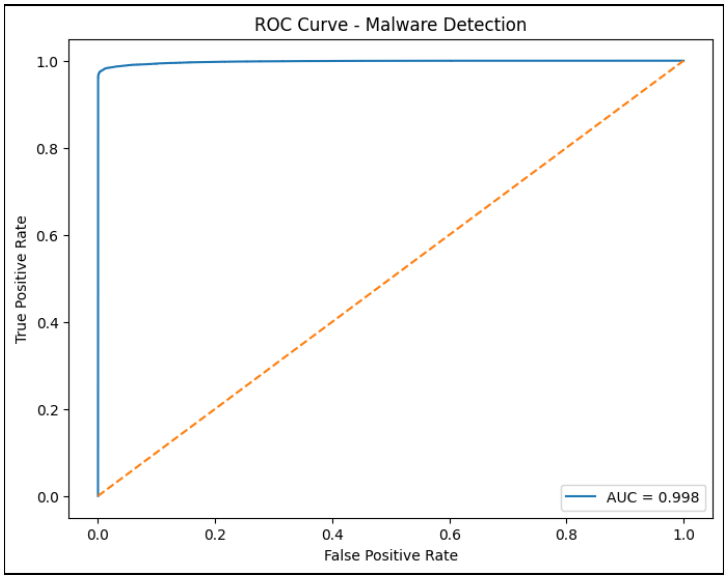


Figure 9.8: ROC Curve of the Binary Model

The curve rises almost vertically near the left edge and stays near the top. The dashed line is the random-guess line. The legend shows AUC = 0.998.

9.11 ROC-AUC Analysis

The ROC curve plots the true positive rate against the false positive rate for all possible thresholds [8]. The notebook calculated the ROC-AUC with roc_auc_score and printed 0.9979768335, which is 0.9980 when rounded to four decimals. A value this close to 1 means that, for almost every pair of one malicious and one benign test record, the model gives the malicious record the higher malicious probability. ROC-AUC does not depend on the threshold of 0.5, so it shows that the model separates the two classes well in general, not only at one cut-off.

9.12 Overall Performance

The binary model achieved a test accuracy of 98.02%, a test loss of 0.0551 and an ROC-AUC of 0.9980. Precision and recall are high for both classes. These results indicate that the binary formulation was more suitable for the primary objective of identifying malicious traffic. They do not mean that the system is perfect: 1,587 malicious test records were still missed, and the results are for one dataset, one architecture and one random split.

10. INTERACTIVE CYBER THREAT DETECTOR

10.1 Purpose

After the binary model was trained and evaluated, we built a small interactive detector to show how the trained model is used on a single record. The detector is built in the same Colab notebook with ipywidgets. Figure 10.1 shows its workflow.

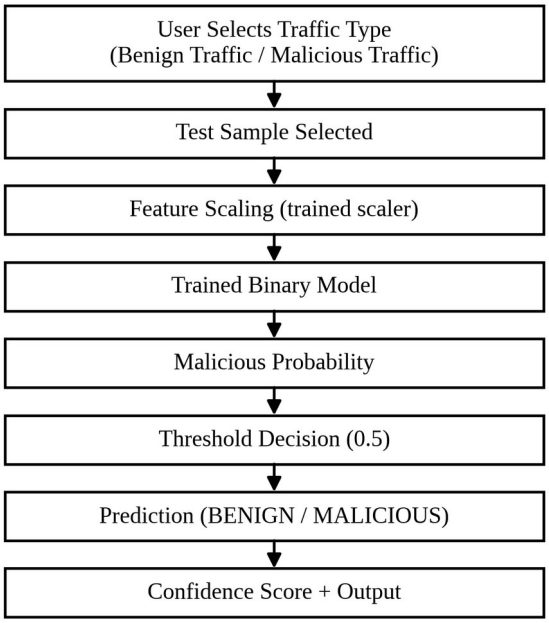


Figure 10.1: Interactive Cyber Threat Detector Workflow

The selected traffic type decides which kind of test sample is used. The sample is scaled, scored by the model, compared with the threshold and displayed with a confidence value.

10.2 Interface Design

The interface was created with ipywidgets and contains four parts: an HTML title (AI Cyber Threat Detector), a dropdown with the options Benign Traffic and Malicious Traffic, a button labelled Analyze Traffic, and an output area in which the result is printed. Figure 10.2 shows the interface after Benign Traffic was selected and the button was clicked.

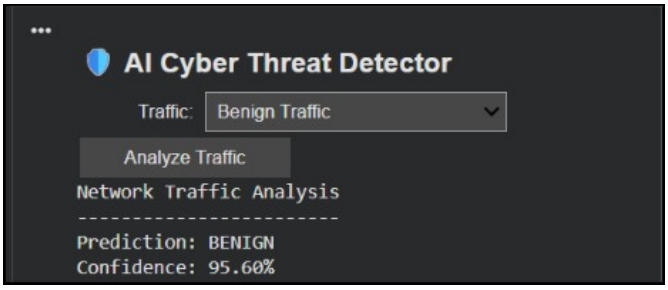


Figure 10.2: Interactive Cyber Threat Detector with a Benign Prediction

Dropdown set to Benign Traffic. The output shows Prediction: BENIGN and Confidence: 95.60%.

Before the widget version we built two simpler versions: a function detect_traffic(sample) that scales a sample, predicts and prints the result, and a text menu that asks the user to enter 1 (benign sample) or 2 (malicious sample) with input(). The menu version is shown in Figure 10.3. The widget replaced the typed menu with a dropdown and a button.

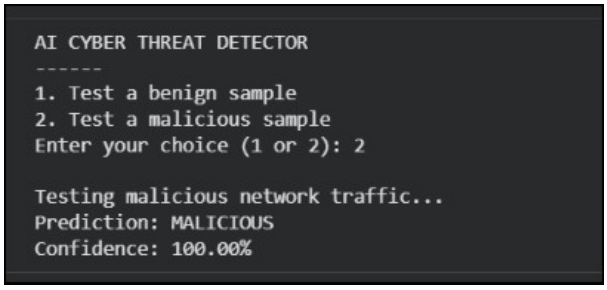


Figure 10.3: Text-Menu Version of the Detector (Earlier Version)

Choice 2 was entered; the sample was classified as MALICIOUS with 100.00% confidence.

10.3 Prediction Workflow

When the Analyze Traffic button is clicked, the notebook performs the following steps:

1. A test sample of the selected type is chosen from the test set.
2. The sample is scaled using the StandardScaler that was fitted on the training data.
3. The binary model predicts the probability that the sample is malicious.
4. A threshold of 0.5 is applied to the probability.
5. The sample is classified as BENIGN or MALICIOUS.
6. The confidence is calculated and the result is printed in the output area.

10.4 Sample Selection

The dropdown values are 0 for Benign Traffic and 1 for Malicious Traffic. The code finds the first test record whose binary label equals the selected value, using np.where(y_binary_test == sample_choice.value)[0][0], and takes the corresponding row of X_test. Therefore the detector uses a fixed example for each type, and it gives the same output each time it is run. It does not capture live traffic and does not accept feature values typed by the user.

10.5 Probability Calculation

The selected row is passed through scaler.transform and then through binary_model.predict. The model returns a single number between 0 and 1, which is the probability that the traffic is malicious.

10.6 Threshold Decision

If the probability is 0.5 or higher, the prediction is MALICIOUS; otherwise it is BENIGN. The same threshold of 0.5 was used to create the predictions for the classification report and confusion matrix in Chapter 9.

10.7 Confidence Score

For a MALICIOUS prediction the confidence is the probability multiplied by 100. For a BENIGN prediction it is (1 - probability) multiplied by 100. The value is printed as a percentage with two decimals. It is the model's own estimate of how likely its answer is, and it is not a guarantee that the answer is right.

10.8 Benign Prediction

For the first benign sample of the test set the detector printed Prediction: BENIGN and Confidence: 95.60% (Figure 10.2). The function version gave the same result, printed as 95.6% (Figure 10.4).

```
# Find a benign sample from the test dataset
benign_position = np.where(y_binary_test == 0)[0][0]
sample = X_test.iloc[[benign_position]]
detect_traffic(sample)

Prediction: BENIGN
Confidence: 95.6 %
```

Figure 10.4: Benign Sample Tested with the detect_traffic Function

The first benign test record is classified as BENIGN with 95.6% confidence.

10.9 Malicious Prediction

For the first malicious sample of the test set the detector printed Prediction: MALICIOUS and Confidence: 100.00% (Figure 10.5). The confidence is shown to two decimals, so a value of 100.00% means the probability was very close to 1, not that the prediction is certain to be correct.

```
Network Traffic Analysis
------------------------
Prediction: MALICIOUS
Confidence: 100.00%
```

Figure 10.5: Malicious Prediction Shown in the Detector Output

The output area after Malicious Traffic was selected and Analyze Traffic was clicked.

The notebook also has a final summary cell that prints the test accuracy (calculated from the model), the ROC-AUC and the precision and recall values. In that cell the ROC-AUC (0.9979768335) and the four precision and recall values were typed in manually; they are the same as the values obtained earlier in the notebook. It also prints the message System Status: Model trained and tested successfully. The detector widget itself prints only the heading, the prediction and the confidence.

10.10 Model Saving

The trained binary model was saved with binary_model.save("cyber_threat_detector.keras"), and the notebook printed Model saved successfully! (Figure 10.6). A saved model file keeps the architecture and the learned weights, so the model can be loaded again in a later session and used for prediction without training it from scratch. We did not reload the file in the notebook, so loading is not shown in this report.

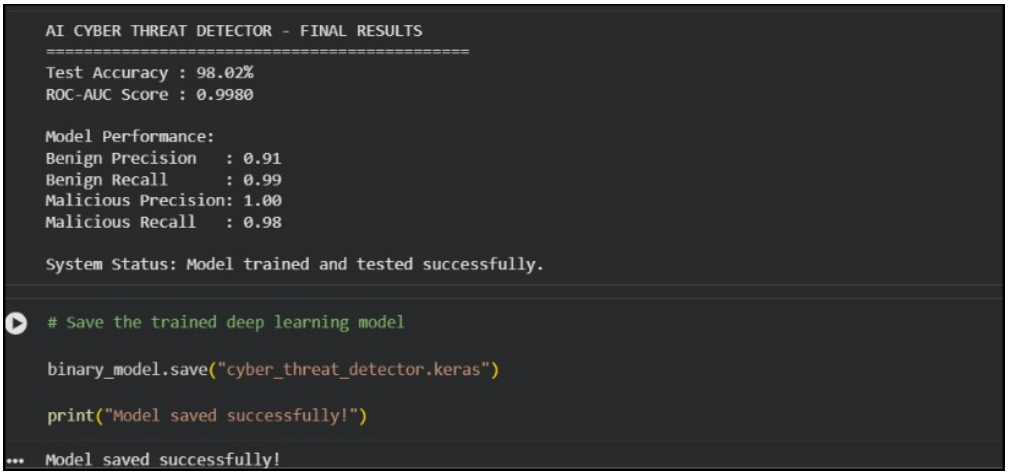


Figure 10.6: Final Results Summary and Model Saving

The final summary cell and the cell that saves the model, followed by the message Model saved successfully!

Two points should be noted. First, the model has not been deployed to any server or application; it was only saved as a file. Second, the fitted StandardScaler was not saved in the notebook. A saved model on its own cannot be used on new unscaled data unless the same scaler is also available, so saving the scaler (for example with joblib) is a necessary step for any later reuse.

11. RESULTS AND PERFORMANCE ANALYSIS

11.1 Summary of Results

Two experiments were carried out on the same prepared data, the same train-test split and the same hidden-layer structure. The five-class experiment reached a test accuracy of 32.72%. The binary experiment reached a test accuracy of 98.02% and an ROC-AUC of 0.9980. Figure 11.1 and Table 11.1 compare the two.

11.2 Multiclass Results

In Experiment 1, Benign (F1-score 0.94) and Gafgyt (0.63) were recognised reasonably, but DarkNexus (0.08), Generic (0.05) and Mirai (0.23) had low F1-scores. The macro-average F1-score was 0.39 and the weighted-average F1-score was 0.37. Most Mirai test records were assigned to DarkNexus or Generic.

11.3 Binary Results

In Experiment 2 the F1-score was 0.95 for Benign and 0.99 for Malicious, with a macro-average F1-score of 0.97. The model missed 1,587 of 68,298 malicious test records and wrongly flagged 92 of 16,524 benign test records.

11.4 Comparison of Experiments

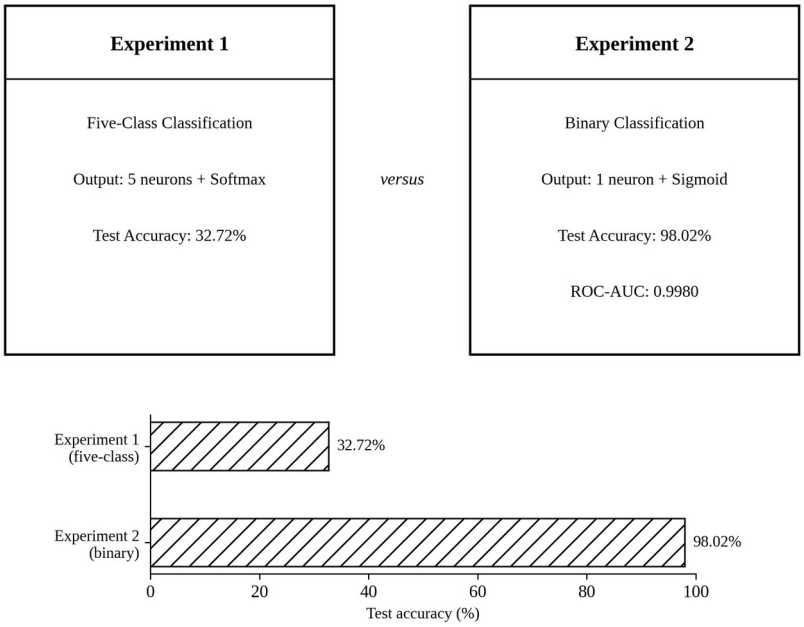


Figure 11.1: Experiment Comparison

Test accuracy of the five-class model (32.72%) and the binary model (98.02%). The binary model also has an ROC-AUC of 0.9980.

Table 11.1: Comparison of Experiments

Aspect	Experiment 1 (five-class)	Experiment 2 (binary)
Task	Benign, DarkNexus, Gafgyt, Generic, Mirai	Benign (0) or Malicious (1)
Output layer	5 neurons, softmax	1 neuron, sigmoid

Aspect	Experiment 1 (five-class)	Experiment 2 (binary)
Loss function	Sparse categorical crossentropy	Binary crossentropy
Trainable parameters	4,805	4,673
Epochs / batch size	10 / 256	10 / 256
Best validation accuracy	38.68% (epoch 2)	98.02% (epoch 8)
Test accuracy	32.72%	98.02%
Test loss	0.9994	0.0551
Macro-average F1-score	0.39	0.97
Weighted-average F1-score	0.37	0.98
ROC-AUC	Not calculated	0.9980

The two accuracy values should not be compared too literally, because the tasks are different: one has five classes and the other has two. The comparison does show that the binary formulation suits our main objective much better, which is to detect malicious traffic. As mentioned in Section 8.8, reading the five-class predictions as benign or malware would give an accuracy of 97.66%, calculated by us from the confusion matrix. This is close to the binary model's 98.02%, which suggests that the benign-versus-malware boundary was learnable in both experiments, and that the family-level distinction was what the first model could not learn.

11.5 Confusion Matrix Analysis

Table 11.2: Error Analysis of the Binary Model (Calculated from the Confusion Matrix)

Measure	Calculation	Value
False positive rate (benign flagged as malicious)	92 / 16,524	0.56%
False negative rate (malicious missed)	1,587 / 68,298	2.32%
Share of predicted-benign records that are malicious	1,587 / 18,019	8.81%
Share of predicted-malicious records that are benign	92 / 66,803	0.14%
Accuracy	(16,432 + 66,711) / 84,822	98.02%

The values in Table 11.2 were calculated by us from the numbers in Table 9.4. They show that the model is very reliable when it says that traffic is malicious (only 0.14% of such predictions were wrong), but about 8.8% of the traffic it labels benign is actually malicious. The benign class is the smaller class, so a few hundred misclassified records have a large effect on its precision.

11.6 ROC-AUC Analysis

The ROC-AUC of 0.9980 for the binary model is close to the maximum of 1.0. Together with the shape of the ROC curve in Figure 9.8 this indicates that the predicted probabilities rank malicious records above benign records almost always. We did not calculate an ROC-AUC for the five-class model.

11.7 Key Findings

- A small network (about 4,700 to 4,800 parameters) trained for 10 epochs was enough to detect malicious versus benign traffic on this data with 98.02% test accuracy.
- The same network could not separate the four malware families reliably (32.72% test accuracy), in particular Mirai from DarkNexus and Generic.
- Benign traffic was recognised well in both experiments.
- Class imbalance strongly influenced the five-class result, and accuracy alone would have hidden the problem; the classification report and confusion matrix showed it.
- Reformulating the problem as a binary task, based on the first experiment, gave a model that fits the main objective of the project.

12. DISCUSSION

12.1 Interpretation of Results

The first experiment tells us that, with these 39 summary features and a small network, it was not possible to separate the malware families from each other, even though benign traffic was separated well. The second experiment shows that, when the family distinction is removed, the two groups can be separated with high accuracy. A test accuracy of 98.02% and an ROC-AUC of 0.9980 mean that, on our test set, the model's decisions were correct for about 98 out of every 100 records and its probabilities ranked the two groups almost perfectly. These numbers describe performance on this test set only. They do not mean that the system would find every malicious flow in other networks.

12.2 Major Findings

- Problem formulation mattered more than the model. The same network gave 32.72% on one formulation and 98.02% on another.
- Preprocessing problems affected the work directly. The infinite values in the Rate column stopped the scaler, and had to be found and handled before any model could be trained.
- Evaluation with only accuracy would not have been enough for either experiment, because of the class imbalance.

12.3 Strengths

- The work follows the complete cycle of data inspection, preprocessing, modelling, evaluation and demonstration.
- A recent (2026) dataset with a documented collection procedure was used [3].
- The test set was kept separate from training, the scaler was fitted on training data only, and infinite values in the test set were filled with training medians.
- Both successful and unsuccessful results are reported, with the decision to reformulate the problem explained.
- The final model is small and trains in under a minute in total (about 3 to 6 seconds per epoch for 10 epochs).

12.4 Practical Relevance

The project shows that network-traffic features can be used to separate benign traffic from IoT malware traffic using a small neural network, and that the choice of problem formulation is important. It is a learning prototype and is not ready for production use. It cannot replace firewalls, intrusion detection systems or other security infrastructure, and it was not tested on real network traffic.

12.5 Lessons Learned During Development

- Check the data for infinite values as well as missing values. Our first scaling attempt failed because we only checked for missing values.
- Do not trust a single number. Accuracy looked acceptable in some places while the confusion matrix showed that one class was almost completely misclassified.
- Class weights help, but they are not a complete solution for imbalance, as the first experiment showed.

- Understand the data structure before splitting it. Only while writing this report did we look at how many distinct samples stand behind the rows, and it changed how we interpret the result.
- Small coding errors are part of the work. For example, an attempt to select a benign sample using .index failed because y_binary_test is a NumPy array, and we replaced it with np.where.

13. CONCLUSION AND FUTURE SCOPE

13.1 Conclusion

In this project we developed a deep learning model for detecting malicious network traffic using the x86 PCAP data of the CIC-YNU-IoTMal 2026 dataset. We inspected and cleaned the data, handled missing and infinite values, split the data, scaled the features and trained two experiments with the same small neural network.

The first experiment classified traffic into five classes and reached a test accuracy of 32.72%. The classification report and confusion matrix showed that the model could not separate the malware families, especially Mirai from DarkNexus and Generic, and that class imbalance was a major difficulty. Based on this result the problem was reformulated as binary classification. The binary model reached a test accuracy of 98.02%, a test loss of 0.0551 and an ROC-AUC of 0.9980, with a precision of 1.00 and a recall of 0.98 for the malicious class. We also built an interactive Cyber Threat Detector with ipywidgets and saved the trained binary model as cyber_threat_detector.keras.

The results show that the binary formulation was more suitable for the primary objective of identifying malicious traffic. The system is a prototype and is not a production security tool.

13.2 Future Scope

- Split the data by Hash (executed sample) so that no sample appears in both training and test sets, and compare the accuracy with the present result.
- Test the model on the ARM, MIPS and MIPSEL PCAP files and on a model trained on one architecture and tested on another.
- Use the SAR and STRACE data of the dataset, together with network traffic, to see whether family classification improves.
- Improve the family-level classification with other methods for imbalance (for example resampling), larger or different networks, and hyperparameter tuning, and report precision and recall for each family.
- Study the 1,587 missed malicious records to find which families they belong to, and examine threshold values other than 0.5 to trade false negatives against false positives.
- Test the model on benign and malicious traffic collected from outside this dataset.
- Save the fitted scaler with the model, reload both from files, and wrap them in a small application or service for testing with new records.
- Repeat training with different random seeds and report the mean and variation of the results.

14. REFERENCES

[1] M. Antonakakis et al., "Understanding the Mirai botnet," in Proc. 26th USENIX Security Symposium, Vancouver, BC, Canada, 2017, pp. 1093–1110.

[2] S. Dadkhah, O. D. Okey, S. A. Maret, Y.-W. Lo, A. Firouzi, R. Kuki, T. Sasaki, K. Yoshioka, T. Ban, S. Ozawa, and A. A. Ghorbani, "CIC-YNU-IoTMal: A comprehensive multilayer dataset for static and dynamic analysis of IoT malware behavior," Information Systems, 2026, Art. no. 102722.

[3] Canadian Institute for Cybersecurity, University of New Brunswick, "CIC-YNU-IoTMal Dataset 2026." [Online]. Available: https://www.unb.ca/cic/datasets/ynu-iot-2026.html (accessed Oct. 17, 2026).

[4] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[5] F. Chollet et al., "Keras," 2015. [Online]. Available: https://keras.io

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2014.

[8] T. Fawcett, "An introduction to ROC analysis," Pattern Recognit. Lett., vol. 27, no. 8, pp. 861–874, 2006.